

Security Corp

Final Consulting Report

Prepared for: M. Alvarez, IT Director, SecureCorp Inc.

Prepared by: Yasasri Valluri, Junior Cybersecurity Consultant

Date: September 15, 2026

CONFIDENTIAL

— For Client and Instructor Use Only

Executive Summary

A security assessment was conducted against SecureCorp's authorized laboratory environment to evaluate its overall security posture, identify weaknesses that could affect business operations, and assess the organization's ability to detect suspicious activity. The assessment covered the customer-facing web application, network services, authentication mechanisms, and security monitoring.

The assessment identified **several significant security weaknesses**. Testing demonstrated that the web application was vulnerable to **SQL Injection, Cross-Site Scripting (XSS), and Command Injection**. Authentication weaknesses were also identified through controlled brute-force testing against web, FTP, and SSH services. These findings show that an attacker could potentially gain unauthorized access, manipulate application behavior, or access sensitive information if similar weaknesses existed in a production environment.

The **highest business risks** are unauthorized access to customer or employee information, compromise of user accounts, manipulation of application or system data, and disruption of services. The presence of multiple exposed network services also increases the organization's attack surface and provides attackers with more opportunities to discover and exploit weaknesses.

These risks could result in data exposure, operational disruption, financial impact, and damage to customer trust.

Security monitoring provided useful visibility during the assessment. **Wazuh detected several of the tested activities**, including SQL Injection attempts and brute-force activity against FTP and SSH. These detections, combined with application and system logs, helped establish a timeline of the testing activity and demonstrated that centralized monitoring can support investigation and response.

Top Recommendations

1. **Remediate web application vulnerabilities** by implementing secure database queries, proper input validation, output encoding, and safe handling of operating-system commands.
2. **Strengthen authentication** through strong passwords, multi-factor authentication, rate limiting, and account lockout or login-delay mechanisms.
3. **Reduce unnecessary network exposure** by disabling services that are not required and restricting access to administrative and database services through firewalls and network segmentation.
4. **Secure remote-access and file-transfer services**, particularly SSH and FTP. SSH access should be restricted and preferably use stronger authentication mechanisms, while FTP should be disabled where unnecessary or replaced with a more secure alternative.
5. **Maintain centralized security monitoring** with Wazuh and continue reviewing alerts and logs so suspicious activity can be detected and investigated promptly.
6. **Establish regular patching and security testing** to identify and address vulnerabilities before they can be exploited.

Overall, SecureCorp should prioritize application security, authentication, attack-surface reduction, and continuous monitoring. The assessment demonstrates that vulnerabilities can be identified and detected in the laboratory environment; addressing these weaknesses before similar systems are exposed to real attackers would significantly reduce potential business impact. Full detail on each recommendation, prioritized, is provided in Section 9.

Scope & Rules of Engagement

3.1 Scope

This security assessment was conducted against an **authorized, isolated laboratory environment created for SecureCorp security testing**. The assessment focused on identifying

vulnerabilities, demonstrating controlled attacks, evaluating security monitoring, and investigating the resulting security events.

The assessment environment consisted of the following systems:

Asset	IP Address	Role
Attacker Environment	172.20.10.5	Security testing and attack simulation
Metasploitable2 Target	172.20.10.3	Vulnerable target system and DVWA application
Wazuh Server	172.20.10.2	Centralized security monitoring and alert investigation

The assessment covered:

- Network and service discovery of the target.
- Web application mapping and security testing of DVWA.
- Controlled brute-force testing against authorized laboratory accounts.
- SQL Injection testing.
- Cross-Site Scripting (XSS) testing.
- Command Injection testing.
- FTP authentication testing.
- SSH authentication testing.
- Wazuh alert detection and investigation.
- Analysis of application and system logs.
- Assessment of security risks and development of remediation recommendations.

The assessment was limited to the systems and activities defined for the laboratory engagement. No external systems, third-party infrastructure, or unauthorized accounts were targeted.

3.2 Rules of Engagement

The following rules were followed throughout the assessment:

1. **Authorization:** All testing was performed only against the designated SecureCorp laboratory systems.
2. **Target Restriction:** The primary target for attack simulations was the Metasploitable2 system at 172.20.10.3.
3. **Controlled Testing:** Attacks were performed in a controlled manner to demonstrate security weaknesses without intentionally causing unnecessary or permanent damage to the laboratory environment.
4. **Credential Testing:** Brute-force activity was restricted to authorized laboratory accounts and small test wordlists.

5. **No Unauthorized Systems:** Systems outside the defined laboratory environment were not scanned, attacked, or accessed.
6. **Evidence Collection:** Relevant screenshots, terminal output, timestamps, application logs, authentication logs, and Wazuh alerts were collected to support assessment findings. The handbook requires findings to be supported by appropriate evidence and recommends recording timestamps and Wazuh detection status.
7. **Monitoring:** Wazuh was used to observe and investigate security events generated during testing. Detection status was recorded as **Automatic, Manual, or Not Visible** for each required activity.
8. **Safety:** Testing was conducted within the authorized lab environment, with care taken to avoid unnecessary disruption to the assessment infrastructure.
9. **Reporting:** Findings were documented together with their potential impact, risk level, supporting evidence, and recommended remediation.

3.3 Assessment Limitations

This assessment represents a **controlled laboratory security assessment** and should not be interpreted as proof that the same vulnerabilities exist in SecureCorp's real production environment.

The results are limited to the systems, services, configurations, and applications available within the assessment environment. The assessment also focused on the attack scenarios specified for the engagement rather than providing a complete security audit of every possible vulnerability.

The final report therefore uses the demonstrated laboratory findings to identify **potential business risks and recommended security improvements**, rather than claiming that an actual production compromise occurred.

4. Asset Inventory & CIA Analysis

4.1 Asset Inventory

The following assets were identified as relevant to SecureCorp's business operations and security posture :

#	Asset	Type	Department / Owner	Purpose	Security Importance
1	Employee laptops and Windows workstations	Hardware	Engineering, HR, Finance,	Used by employees to access company	Compromise could expose credentials,

#	Asset	Type	Department / Owner	Purpose	Security Importance
			Customer Support	systems and data	business information, and sensitive data
2	Ubuntu web server	Hardware / Software	Engineering	Hosts the customer-facing web application	Compromise could expose application data, credentials, or the underlying server
3	Customer portal web application	Software	Engineering / Customer Support	Provides customer login, account access, and support functionality	Vulnerabilities could allow unauthorized access, data exposure, or manipulation
4	Internal company website	Software	Engineering	Provides internal company information and services	Unauthorized modification could disrupt operations or affect organizational trust
5	Customer database	Data	Engineering / Customer Support	Stores customer-related information	Unauthorized disclosure or modification could cause privacy, financial, and reputational impact
6	Employee database	Data	HR / Engineering	Stores employee-related information	Unauthorized access could expose sensitive employee information
7	Employee accounts and credentials	Data	All departments / IT Administration	Used to authenticate employees and access company resources	Stolen credentials could allow attackers to impersonate legitimate users

#	Asset	Type	Department / Owner	Purpose	Security Importance
8	Internet-facing connection	Connectivity	Engineering / IT Administration	Provides external connectivity to company services	Increases the organization's externally accessible attack surface
9	Customer support tickets	Data	Customer Support	Contains customer requests and technical/support information	Unauthorized access could expose confidential customer or technical information

4.2 CIA Analysis

The **CIA Triad — Confidentiality, Integrity, and Availability** — was used to evaluate the security requirements of the major assets.

Asset	Confidentiality	Integrity	Availability
Customer Database	Customer information must only be accessible to authorized users and applications.	Customer records must not be modified or deleted without authorization.	The database must remain available to support customer services and business operations.
Employee Database	Employee records must be protected from unauthorized disclosure.	Employee information must remain accurate and protected from unauthorized changes.	HR and authorized management must be able to access the database when required.
Customer Portal	Customer credentials, account information, and support information must be protected.	Unauthorized users must not be able to modify accounts or application data.	The portal should remain accessible so customers can use the organization's services.
Ubuntu Web Server	Server credentials, configuration, and application files must be protected.	System and application files must not be altered without authorization.	The server must remain operational to provide the customer portal.
Employee Accounts & Credentials	Authentication information must remain confidential.	Account permissions and credentials must not be changed without authorization.	Employees must be able to authenticate and access the resources required for their work.

Asset	Confidentiality	Integrity	Availability
Customer Support Tickets	Ticket contents may contain customer and technical information and must be restricted to authorized personnel.	Ticket records should remain accurate and protected from unauthorized modification.	Support staff should be able to access tickets when responding to customers.

CIA Prioritization

Based on the identified assets:

- **Customer Database:** High confidentiality, high integrity, high availability.
- **Employee Database:** High confidentiality and integrity, with availability required for business operations.
- **Customer Portal:** High availability, confidentiality, and integrity because it is customer-facing.
- **Ubuntu Web Server:** High integrity and availability, with confidentiality also important because it may contain credentials and application configuration.
- **Employee Accounts & Credentials:** High confidentiality and integrity because compromised credentials can enable unauthorized access.
- **Customer Support Tickets:** High confidentiality and integrity, with availability required for effective customer support.

This asset and CIA analysis provides the foundation for the risk assessment and prioritization of security controls in the following sections.

5. Risk Assessment

A risk assessment was performed to identify the major threats to SecureCorp's assets and evaluate their potential business impact. The assessment considered the affected asset, likely attack method, underlying weakness, potential impact, and overall risk level.

Asset	Threat	Vulnerability	Impact	Risk Rating	Recommendation
Customer portal	SQL Injection	User input is not properly validated or parameterized	An attacker could access, modify, or extract database information, potentially exposing customer data.	Critical	Use parameterized queries/prepared statements, strict input validation, least-privilege database accounts, and regular application security testing.
Customer portal	Brute-force attack	Weak authentication controls and	An attacker could guess customer	High	Implement MFA, account lockout/rate

Asset	Threat	Vulnerability	Impact	Risk Rating	Recommendation
		lack of rate limiting	credentials and gain unauthorized access to accounts.		limiting, strong password policies, and monitoring for repeated failed logins.
Customer portal	Cross-Site Scripting (XSS)	User-controlled input is rendered without appropriate output encoding	Attackers could execute malicious scripts in another user's browser and potentially steal session information or manipulate the interface.	High	Apply context-aware output encoding, input validation, Content Security Policy, and secure cookie settings.
Ubuntu web server	Command Injection	Application input may be passed to operating-system commands without proper validation	Successful exploitation could allow arbitrary commands to execute on the server and potentially lead to full server compromise.	Critical	Avoid passing user input to shell commands, use safe APIs, validate input, run services with least privilege, and isolate applications.
SSH service	Credential brute force	SSH accepts repeated authentication attempts and may use weak/default credentials	Attackers could gain unauthorized shell access to the server.	High	Use strong unique credentials or SSH keys, disable unnecessary password authentication, implement rate limiting/fail2ban, and monitor authentication failures.
FTP service	Credential brute force	FTP authentication may allow repeated login attempts and weak credentials	Successful access could allow unauthorized users to access or modify files.	High	Disable FTP if unnecessary, use SFTP instead, enforce strong credentials, restrict access, and monitor authentication attempts.

Asset	Threat	Vulnerability	Impact	Risk Rating	Recommendation
Employee user accounts	Phishing / credential theft	Users may be tricked into providing credentials to attackers	Stolen credentials could allow attackers to impersonate employees and access internal systems.	High	Implement MFA, phishing-resistant authentication, security awareness training, email filtering, and login monitoring.
Internet-facing connection	Network scanning and exploitation	Publicly reachable services increase the attack surface	Attackers could identify vulnerable services and attempt exploitation, potentially leading to server compromise.	High	Minimize exposed services, use firewall rules, segment networks, regularly scan for vulnerabilities, and monitor network activity.

5.1 Risk Prioritization

Critical Risks

SQL Injection — Customer Portal

SQL Injection represents a critical risk because exploitation of an insecure database query could allow an attacker to interact with backend data beyond the permissions intended by the application. This could potentially result in unauthorized disclosure, modification, or deletion of customer information.

Command Injection — Ubuntu Web Server

Command Injection is critical because successful exploitation could allow an attacker to execute unauthorized commands on the underlying server. A compromised web server could subsequently affect application availability, data confidentiality, and system integrity.

High Risks

The assessment identified several high-risk areas, including:

- **Web application brute force**, which could lead to unauthorized customer account access.
- **XSS**, which could allow malicious content to execute in a user's browser.
- **SSH brute force**, which could result in unauthorized remote access.
- **FTP brute force**, which could provide unauthorized access to files.
- **Credential theft**, which could allow attackers to impersonate legitimate employees.

- **Excessive network exposure**, which increases the number of services available for potential attack.
- **Insufficient monitoring**, which could delay detection and response to malicious activity.

5.2 Risk Treatment Priorities

The recommended priority for addressing these risks is:

1. **Immediately remediate critical web application vulnerabilities**, particularly SQL Injection and Command Injection.
2. **Strengthen authentication controls** using strong passwords, MFA, rate limiting, and account lockout/delay mechanisms.
3. **Reduce unnecessary network exposure** by disabling services that are not required and restricting access to sensitive services.
4. **Harden SSH and replace or disable insecure FTP where possible.**
5. **Maintain centralized logging and Wazuh monitoring** to improve detection and investigation.
6. **Establish regular patching and security testing** to identify vulnerabilities before they can be exploited.

The risk assessment connects the asset and CIA analysis to the findings demonstrated during the assessment and provides the basis for the recommendations presented later in the report.

6. Methodology

The security assessment followed a structured

Discovery → Testing → Detection → Investigation → Assessment

approach. All activities were performed within the authorized lab environment, with the attacker system, target system, and Wazuh monitoring server kept within the defined scope.

6.1 Lab Environment

The assessment used three primary systems:

System	IP Address	Purpose
Attacker Environment (Linux)	172.20.10.5	Network discovery and controlled security testing
Metasploitable2 / DVWA Target	172.20.10.3	Vulnerable target for web, service, and authentication testing
Wazuh Server	172.20.10.2	Centralized security monitoring, alerting, and log investigation

The Metasploitable2 system used the legacy **OSSEC HIDS v2.8.3** agent because of its old operating system and architecture. The Wazuh server was used as the central monitoring and

analysis platform.

6.2 Wazuh Deployment and Monitoring

Wazuh was deployed as the central security monitoring component of the lab. The Wazuh Manager operated on `172.20.10.2`, while the Metasploitable2 target was configured to forward security-related events to the manager.

The monitoring setup covered:

- Authentication activity
- SSH login failures
- FTP authentication failures
- Apache web-server activity
- DVWA-related HTTP requests
- Security rules and alerts generated by Wazuh
- Correlation of Wazuh events with target-side logs

Because the Metasploitable2 system uses the legacy OSSEC agent, its communication configuration was adjusted to work with the monitoring server. The manager was configured to accept the required secure monitoring traffic, including **UDP/1514**, which was observed during troubleshooting.

The Wazuh deployment was then used during the attack phase to determine whether each activity was detected **automatically, manually, or not visibly detected**.

6.3 Discovery Phase

The first step was to establish a baseline of the target before performing security tests.

Network discovery was performed from the attacker environment using Nmap:

```
nmap -sV 172.20.10.3
```

This identified the target as reachable and revealed multiple exposed services, including:

- FTP — TCP/21
- SSH — TCP/22
- HTTP — TCP/80
- SMB — TCP/139 and 445
- MySQL — TCP/3306
- PostgreSQL — TCP/5432

- VNC — TCP/5900
- Apache Tomcat — TCP/8180

The HTTP service was then used to access the DVWA application and identify the available testing modules.

6.4 Web Application Mapping

The DVWA application was mapped before testing individual vulnerabilities. The following modules were identified as relevant to the assessment:

1. **Brute Force**
2. **SQL Injection**
3. **XSS**
4. **Command Injection**

Testing was performed against the application's **Low** security level as required by the assessment procedure. Inputs, payloads, responses, timestamps, and screenshots were recorded as evidence.

6.5 Controlled Security Testing

The testing phase reproduced common attack scenarios within the authorized lab.

Web Brute Force

A small credential list was used against the DVWA Brute Force module. The activity successfully demonstrated that the lab account `admin` could be compromised using the weak password `password`.

SQL Injection

The DVWA SQL Injection functionality was tested using an injection payload against an injectable input field. Wazuh subsequently generated an alert associated with SQL injection activity.

Cross-Site Scripting

A reflected XSS payload was submitted through the DVWA XSS module. Successful execution was verified through the browser-generated alert popup.

Command Injection

The DVWA Command Injection module was accessed and tested using a harmless command chain. The test was recorded in the Apache access logs. Where command output was not directly evidenced, the activity was treated as a **module access/test** rather than claiming confirmed operating-system command execution.

FTP Brute Force

Controlled authentication attempts were performed against the FTP service on TCP/21 using a small authorized credential list. Wazuh detected the rapid sequence of failed authentication attempts.

SSH Brute Force

Controlled SSH authentication attempts were performed against the designated lab account. Multiple failed authentication attempts were recorded in the authentication logs and detected by Wazuh.

6.6 Evidence Collection

Evidence was collected throughout the assessment rather than only after testing was completed.

The evidence included:

- Nmap scan output
- Terminal output
- Screenshots of DVWA tests
- Browser results such as the XSS popup
- Apache access logs
- SSH authentication logs
- FTP/vsftpd logs
- Wazuh alerts
- Timestamps and source/target IP addresses

Each activity was associated with its relevant evidence and recorded in the assessment timeline.

6.7 Detection and Investigation

After each security test, the corresponding monitoring data was reviewed.

The investigation process was:

1. Identify the timestamp of the attack.
2. Identify the source and target IP addresses.
3. Search Wazuh for related alerts.
4. Identify the applicable Wazuh rule where available.
5. Examine the target-side log for supporting evidence.
6. Compare timestamps between Wazuh and application/system logs.

7. Determine whether the activity was detected automatically, manually, or was not visible in Wazuh.
8. Assess the security impact and document the finding.

During the investigation, several Wazuh rules were observed, including:

- **Rule 31104** — SQL injection-related detection
- **Rule 11451** — FTP authentication failure/brute-force detection
- **Rule 5760** — SSH authentication failure/brute-force detection

This allowed the assessment to correlate attack activity with centralized monitoring rather than relying only on the attacker's terminal output.

6.8 Risk Assessment and Recommendations

The final stage compared the demonstrated weaknesses with the assets and CIA requirements identified during Week 1.

Each finding was assessed according to:

- Affected asset/service
- Attack type
- Evidence collected
- Potential impact
- Risk severity
- Detection capability
- Recommended remediation

The recommendations were prioritized around the most significant weaknesses identified during testing, particularly **web-application security, authentication protection, exposed network services, SSH/FTP security, and continuous monitoring**.

6.9 Methodology Summary

Overall, the assessment followed a repeatable process:

Baseline → Discover → Map → Test → Collect Evidence → Detect → Investigate → Assess Risk → Recommend Remediation

This approach ensured that the Week 2 attack activities were directly connected to the Week 3 detection and investigation work and ultimately to the Week 4 risk treatment and reporting.

The screenshot shows the 'Agents (1)' page in the Wazuh dashboard. It includes a search bar, a table with agent details, and a 'WQL' button. The table has columns for ID, Name, IP address, Group(s), Operating system, Cluster node, Version, Status, and Actions. One agent is listed: ID 001, Name metas-2, IP address 172.20.10.3, Group(s) default, Operating system -, Cluster node node01, Version -, Status active, and Actions. The page also shows 'Rows per page: 10' and a pagination control.

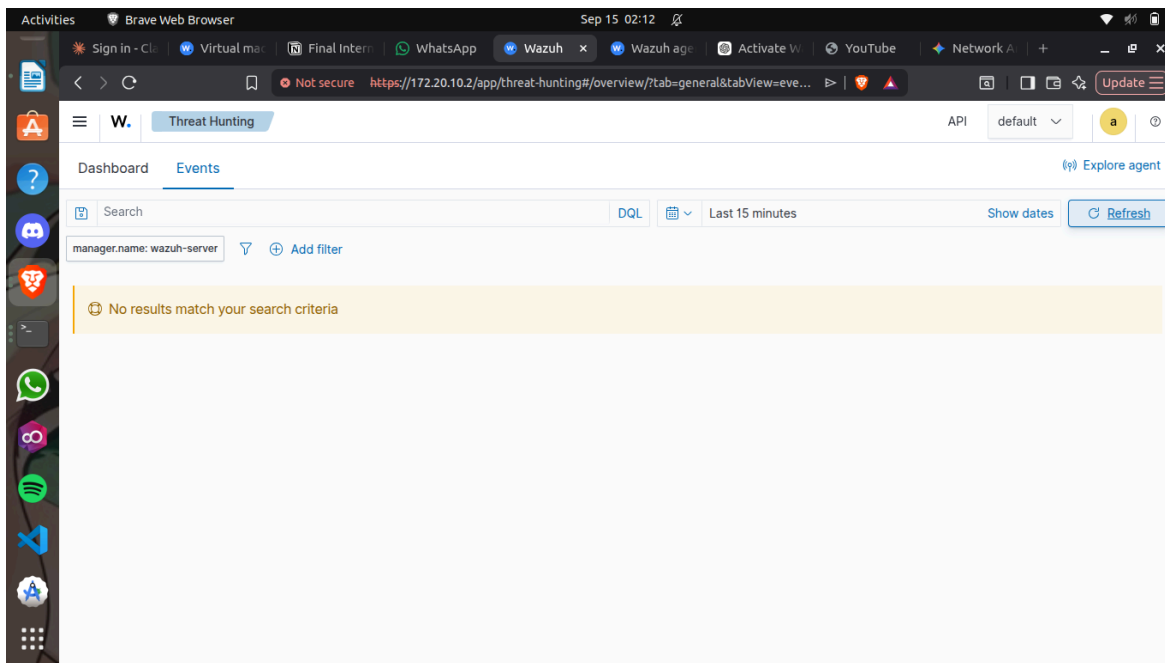
ID	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
001	metas-2	172.20.10.3	default	-	node01	-	active	

The above screenshot clearly shows the deployment of wazuh agent.

7. Attack Simulation Findings

This section documents the controlled attack simulations performed against the authorized Metasploitable2/DVWA target ([172.20.10.3](#)) from the attacker environment ([172.20.10.5](#)). The activities correspond to **Tasks 2.3–2.8** and were conducted at the required low-security configuration where applicable.

All testing was performed within the defined lab scope. The results demonstrate security weaknesses in the deliberately vulnerable environment and should not be interpreted as evidence of an actual production compromise.



The above screenshot is taken and from the image, you can see that no events are there since last 15 minutes at [2:12 am](#). This screenshot is taken so that we can understand how the events look when an attack is performed. We'll understand this in Week 3.

Network Discovery :

```

yasasri@T0g3p1: ~/Week2/2.1_network_discovery
yasasri@T0g3p1:~/Week2/2.1_network_discovery$ nmap -sV 172.20.10.3 | tee ~/Week2/2.1_network_discovery/nmap_scan.txt
Starting Nmap 7.80 ( https://nmap.org ) at 2026-09-15 02:19 IST
Nmap scan report for 172.20.10.3
Host is up (0.000080s latency).
Not shown: 977 closed ports
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rshexecd
513/tcp   open  login          OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi       GNU Classpath grmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100003)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  X11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8009/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.57 seconds
yasasri@T0g3p1:~/Week2/2.1_network_discovery$

```

The used command is :

```
nmap -sV 172.20.10.3 | tee ~/Week2/2.1_network_discovery/nmap_scan.txt
```

We can see that a lot of ports are open.

Target: 172.20.10.3

Source: 172.20.10.5

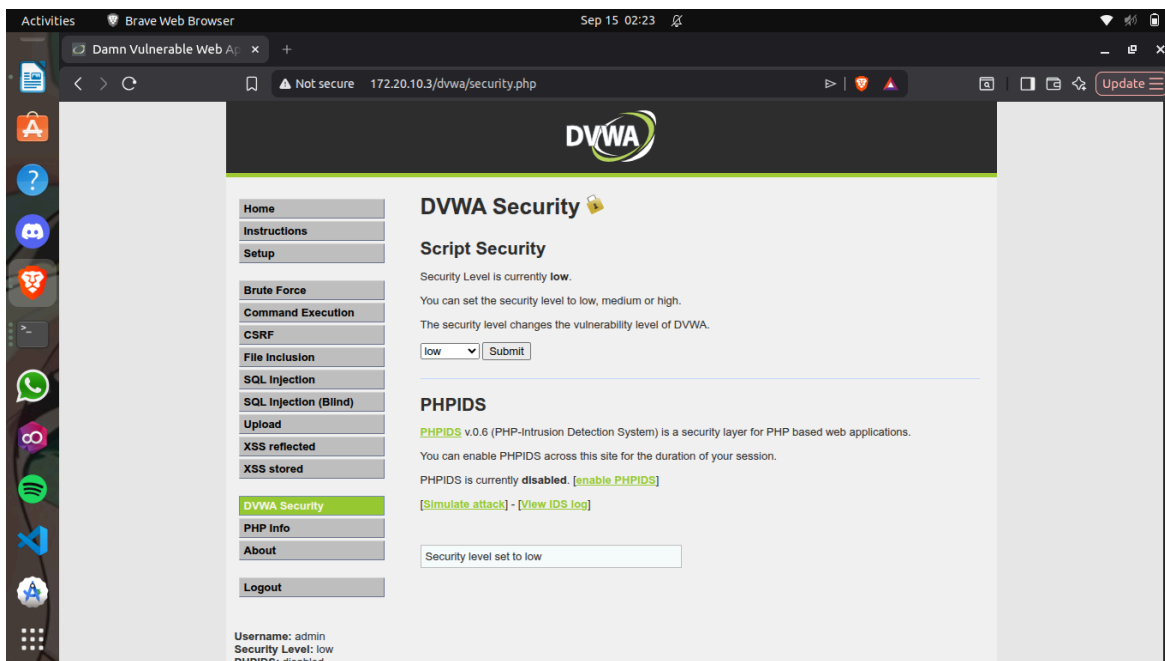
Method: Nmap service/version detection

Network reconnaissance was performed before exploitation to identify reachable services and their corresponding attack surfaces.

The following table gives details about the services and the port numbers and small information about them.

Port	Service	Version	What it suggests
21	FTP	vsftpd 2.3.4	FTP file-transfer service; authentication attack surface
22	SSH	OpenSSH 4.7p1	Remote secure shell login; SSH authentication attack surface
23	Telnet	Linux telnetd	Remote terminal access; authentication attack surface
25	SMTP	Postfix smtpd	Mail transfer service
53	DNS	ISC BIND 9.4.2	DNS/name-resolution service
80	HTTP	Apache 2.2.8	Web server; DVWA/customer-portal attack surface
111	RPCbind	RPC #100000	RPC service mapping

Port	Service	Version	What it suggests
139	NetBIOS	Samba	SMB/Windows file and network-sharing service
445	SMB	Samba	SMB file/network-sharing service
512	rexec	netkit-rsh rexecd	Remote command execution/login service
513	login	—	Remote login service
514	TCP wrapped	—	TCP-wrapped service
1099	Java RMI	GNU Classpath	Java remote method invocation service
1524	bindshell	Metasploitable	Exposed shell service
2049	NFS	NFS 2-4	Network file-system service
2121	FTP	ProFTPD 1.3.1	Alternate FTP service
3306	MySQL	5.0.51a	Database service
5432	PostgreSQL	8.3.x	Database service
5900	VNC	Protocol 3.3	Remote graphical desktop access
6000	X11	—	X Window graphical display service
6667	IRC	UnrealIRCd	IRC chat service
8009	AJP13	Apache Jserv	Tomcat application-server connector
8180	HTTP	Apache Tomcat	Java web application server



Using the target vm's ip `172.20.10.3` , I've opened `dvwa` and set the security level to low as instructed.

The following are the important things that we've focus on later

Module	Purpose	Input	Planned test
Brute Force	Authentication testing	Username/password	Repeated login attempts
SQL Injection	Database query manipulation	User ID/input field	SQL injection
XSS	Browser-side script execution	Text/input field	Alert-box payload
Command Injection	OS command execution	Host/IP input	Harmless command chaining

For all the six attacks,

Target ip : 172.20.10.3

Attacker ip : 172.20.10.5

Attack Simulation 1 — Brute Force

```

yasasri@T0g3p1:~$ cat pass.txt
admin
password
123456
root
buhaa
hui
haha
hehe
huhu
msfadmin
youcantfindme
yasasri@T0g3p1:~$ hydra -l admin -P pass.txt 172.20.10.3 http-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:Username and/or password incorrect.:H=Cookie: PHPSESSID=a1d88ef1780e834b56869387f83c23c3; security=low"
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes
(this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-15 02:57:15
[DATA] max 11 tasks per 1 server, overall 11 tasks, 11 login tries (L:1/p:11), ~1 try per task
[DATA] attacking http-get-form://172.20.10.3:80/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:Username and/or password incorrect.:H=Cookie: PHPSESSID=a1d88ef1780e834b56869387f83c23c3; security=low
[80][http-get-form] host: 172.20.10.3 login: admin password: password
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-15 02:57:16
yasasri@T0g3p1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp44s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 4c:d7:17:6f:4b:fe brd ff:ff:ff:ff:ff:ff
3: wlp43s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 3c:55:76:9b:fa:57 brd ff:ff:ff:ff:ff:ff
    inet 172.20.10.5/28 brd 172.20.10.15 scope global dynamic noprefixroute wlp43s0
        valid_lft 2711sec preferred_lft 2711sec
    inet6 2401:4900:647a:9164:f016:d54:679e:6986/64 scope global temporary dynamic

```

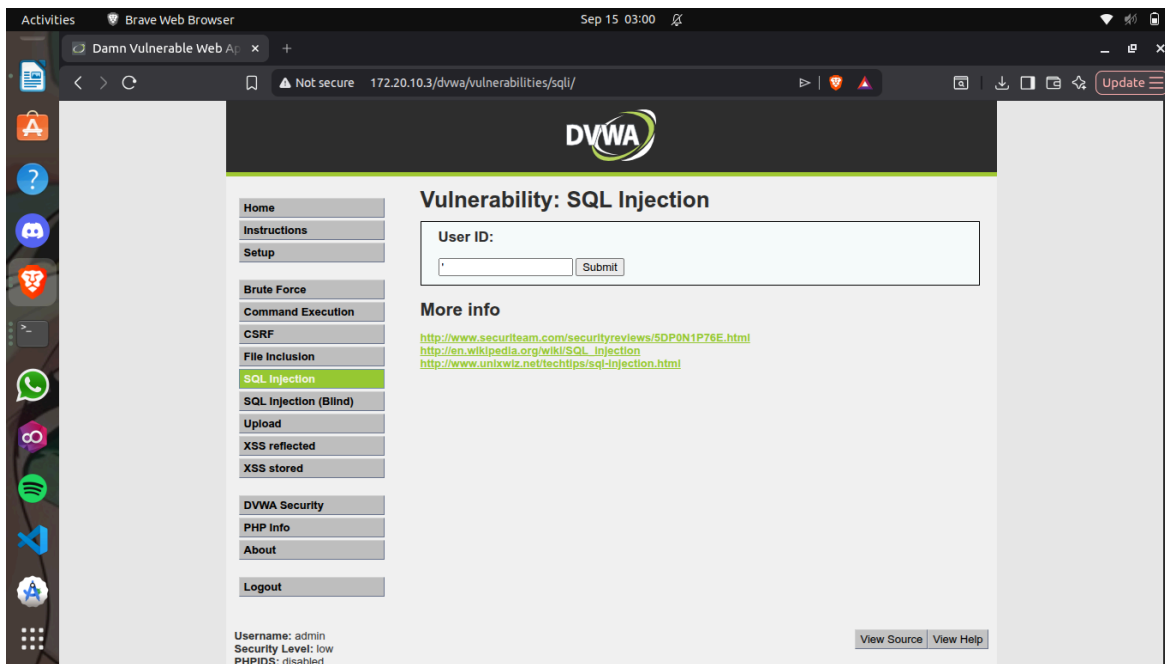
I've created a small wordlist named `pass.txt` . Then i've used hydra to brute force and gave the `url` required and the `PHPSESSID` which is the cookie. You can see that hydra found the correct password which is `password` .

Time Stamp : September 15th 2:58 am

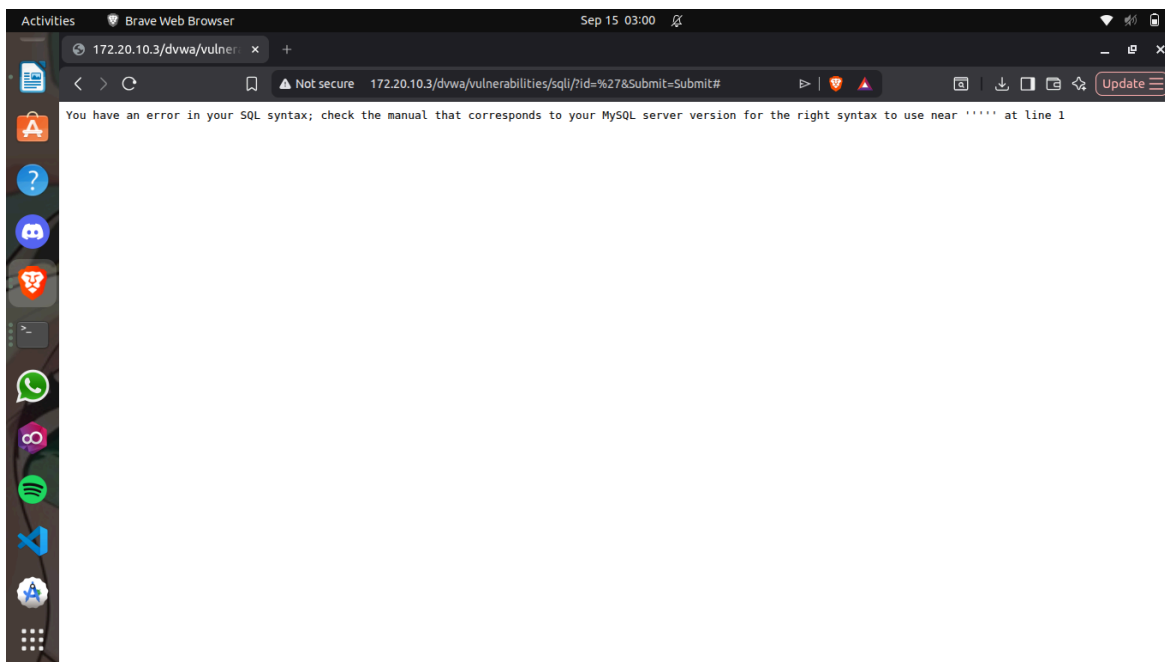
Learning Outcome : How brute-force and credential attacks work in practice, and why weak or reused passwords are a real business risk.

Fix : Implement strong password policies, MFA, login rate limiting, and progressive account lockout/delay mechanisms. Authentication events should be centrally monitored through Wazuh to detect repeated failures and suspicious login patterns. Unnecessary exposed authentication services such as FTP should be disabled or replaced with more secure alternatives.

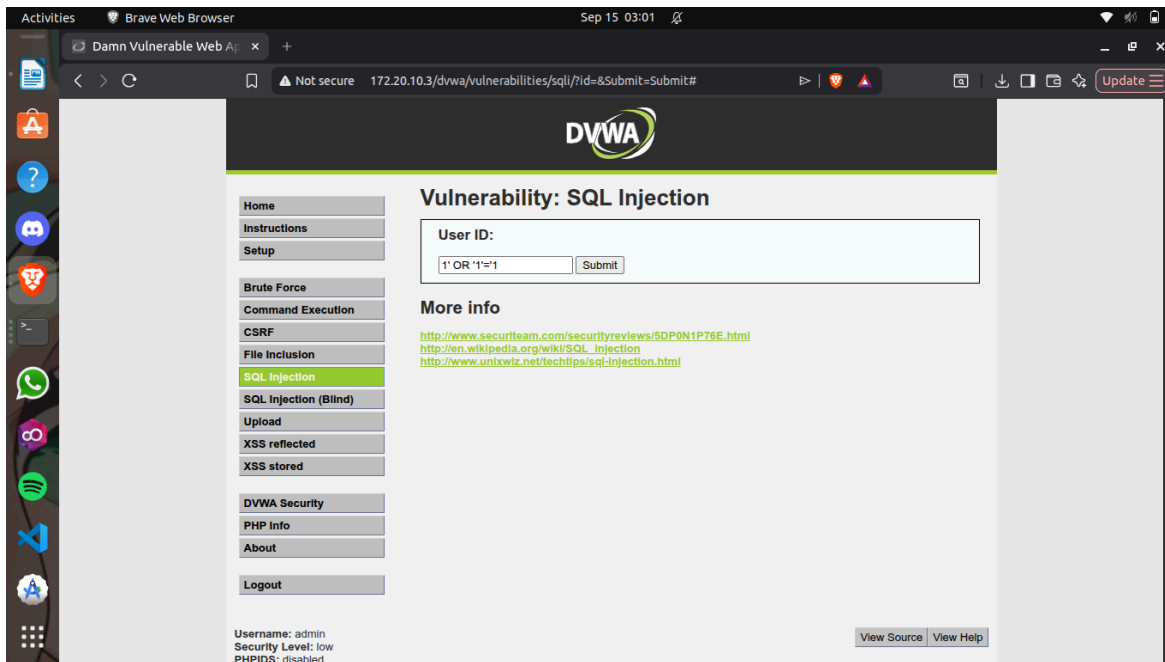
Attack Simulation 2 — SQL Injection



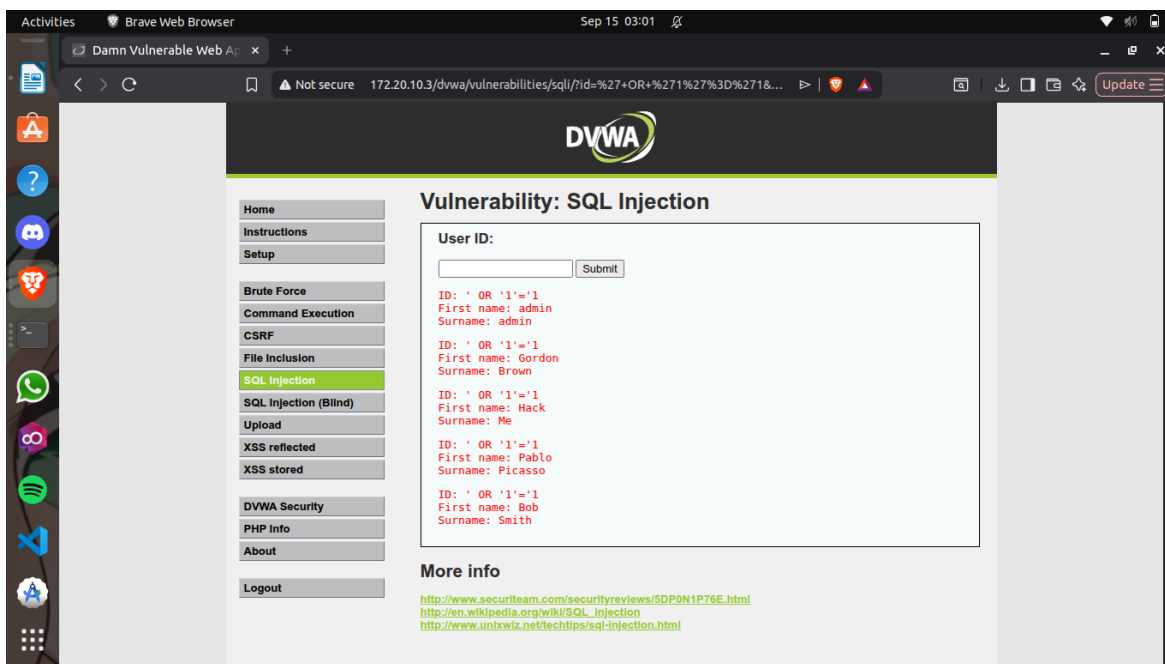
I've just entered ' in the input statement which closes the ' that's there in the query. Submitting this gave me an error.



So I've used the standard and most used SQL Injection which forces it to show all details bcuz the second statement `1==1` is always true.



Hence submitting that gave the details of all users.



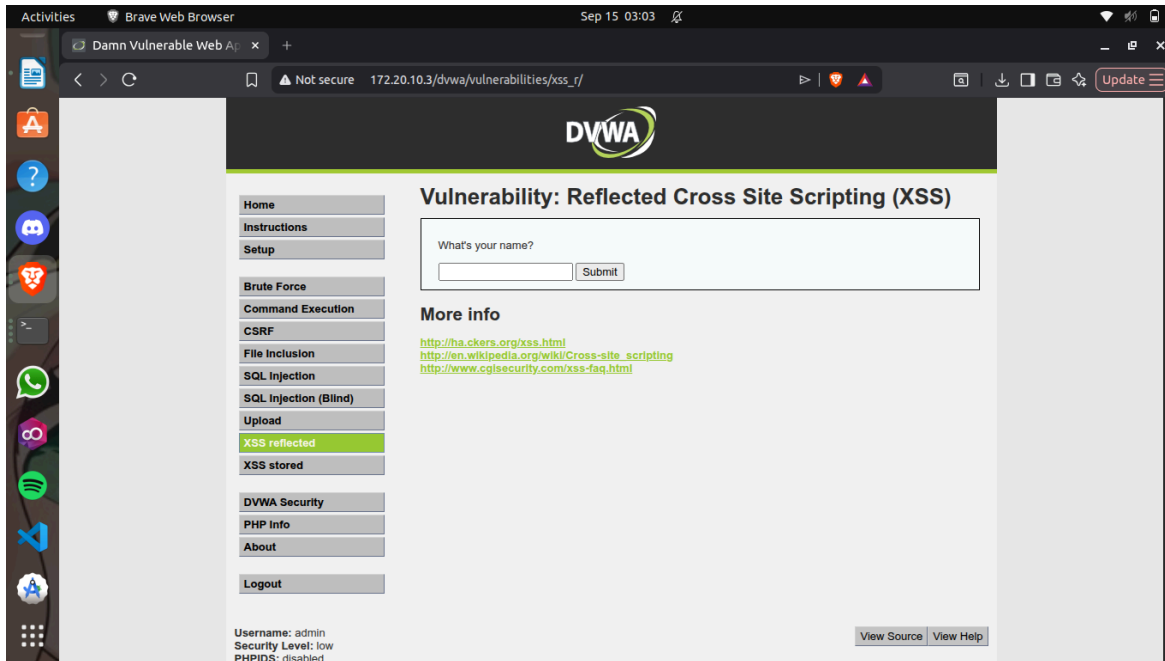
Time Stamp : September 15th 3:01 am

Learning Outcome: How unsanitized input becomes a data-disclosure vulnerability.

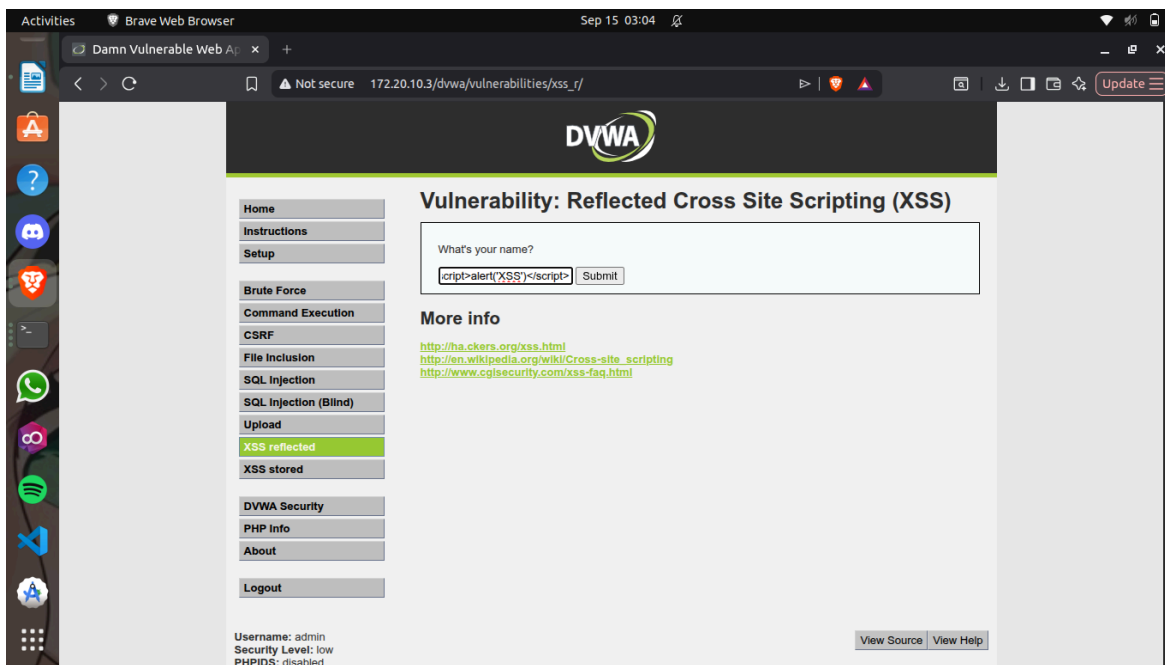
Fix : Implement parameterized queries/prepared statements for all database operations and avoid constructing SQL queries through direct concatenation of user-controlled input. Apply server-side input validation as an additional control, restrict database accounts using least privilege, and

prevent detailed database errors from being exposed to users. Continue Wazuh monitoring for suspicious SQL injection patterns.

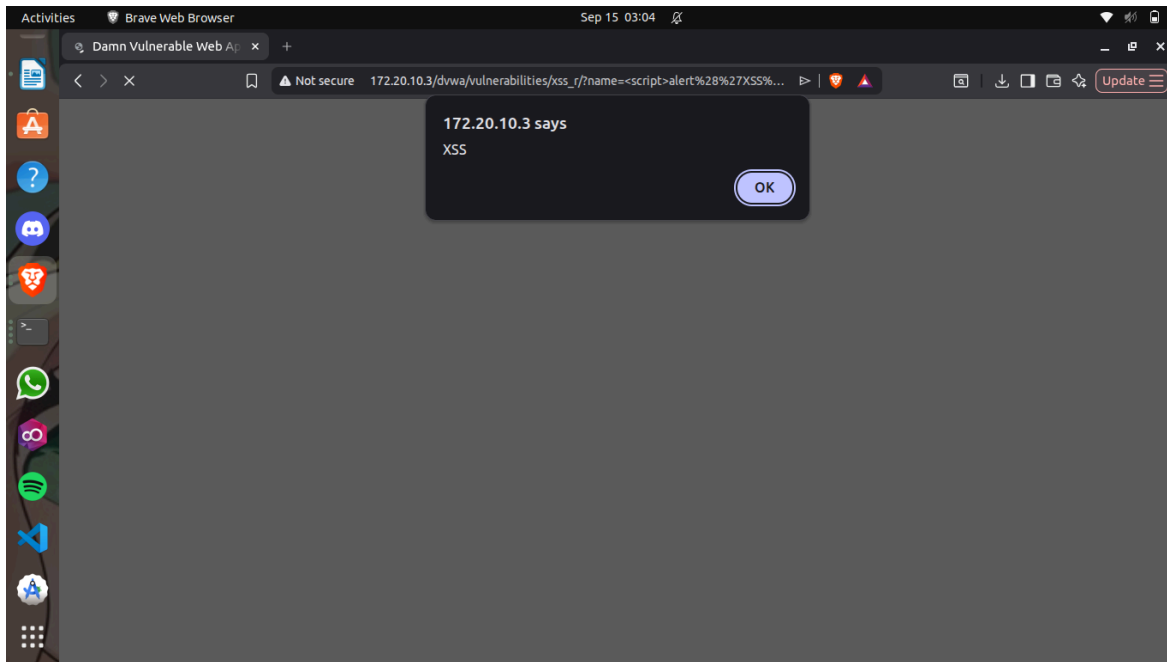
Attack Simulation 3 — Cross-Site Scripting (XSS)



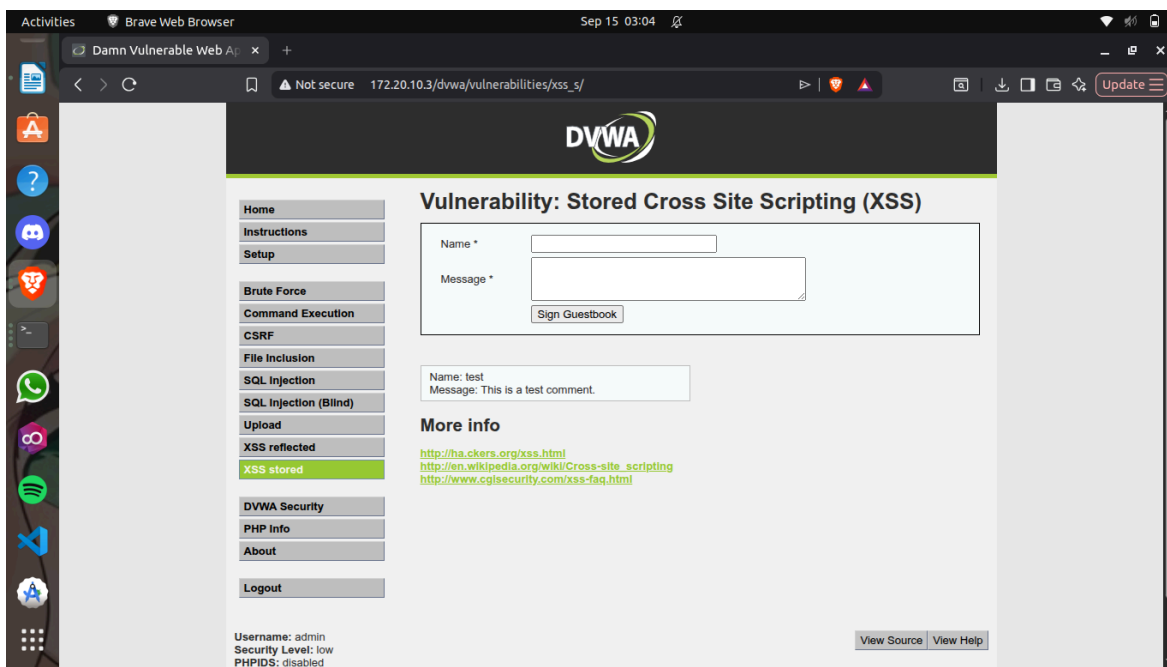
Its asking for `name`.

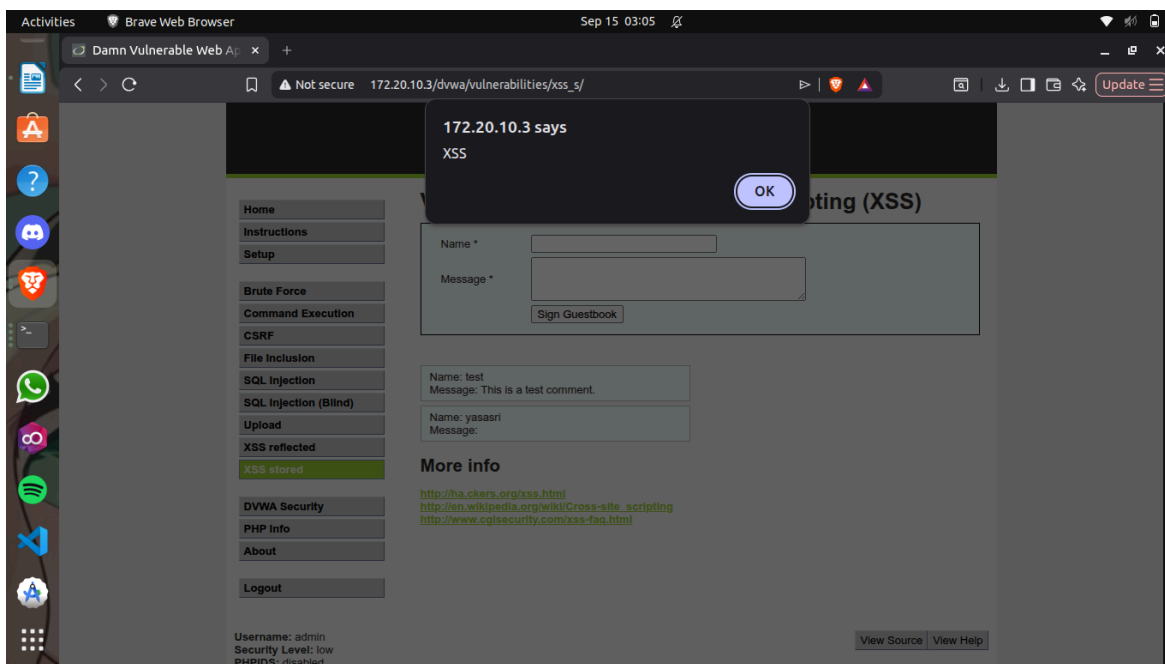
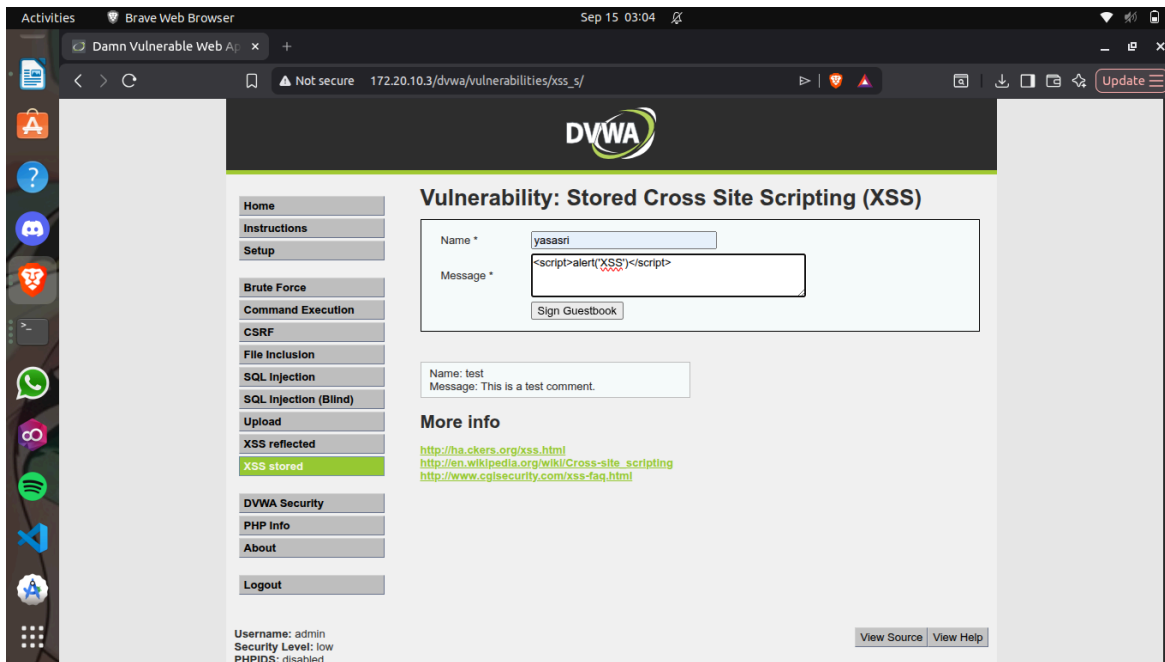


Since this is XSS vuln, we can simply inject a script which creates an alert box saying `XSS`. Submitting that script will execute that script and we'll get an alert box.



Stored XSS :





Injecting the same script before in **message** box triggered the alert.

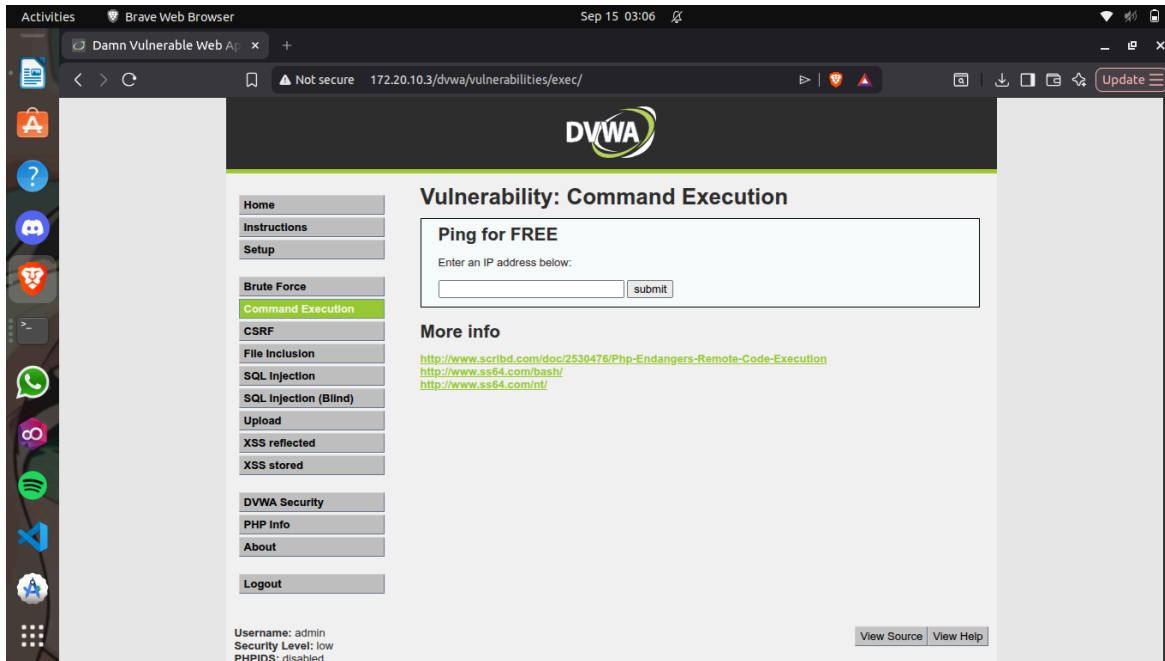
Time Stamp : **September 15th 3:04 am**

Learning Outcome : Difference between reflective and stored XSS and how input sanitization matters.

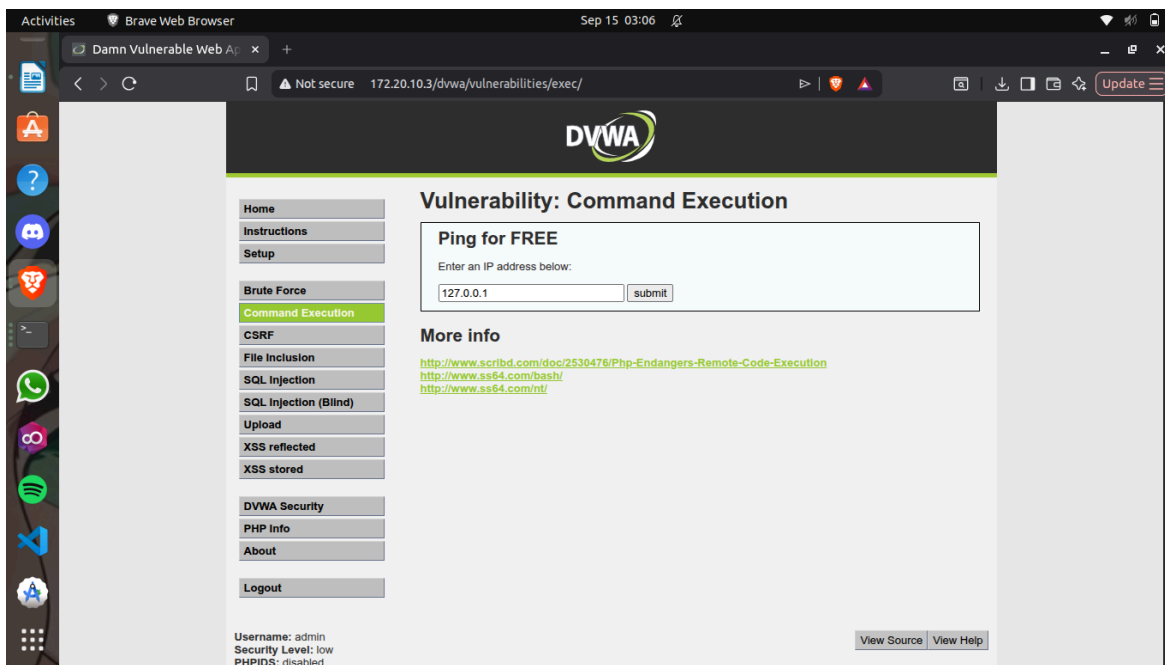
Fix : Implement proper server-side input validation and context-aware output encoding to prevent user-controlled data from being interpreted as executable JavaScript. Use a suitable Content Security Policy (CSP) as an additional security layer, and configure cookies with appropriate

security attributes such as `HttpOnly` and `Secure`. Regularly test web applications for XSS vulnerabilities during development and security assessments.

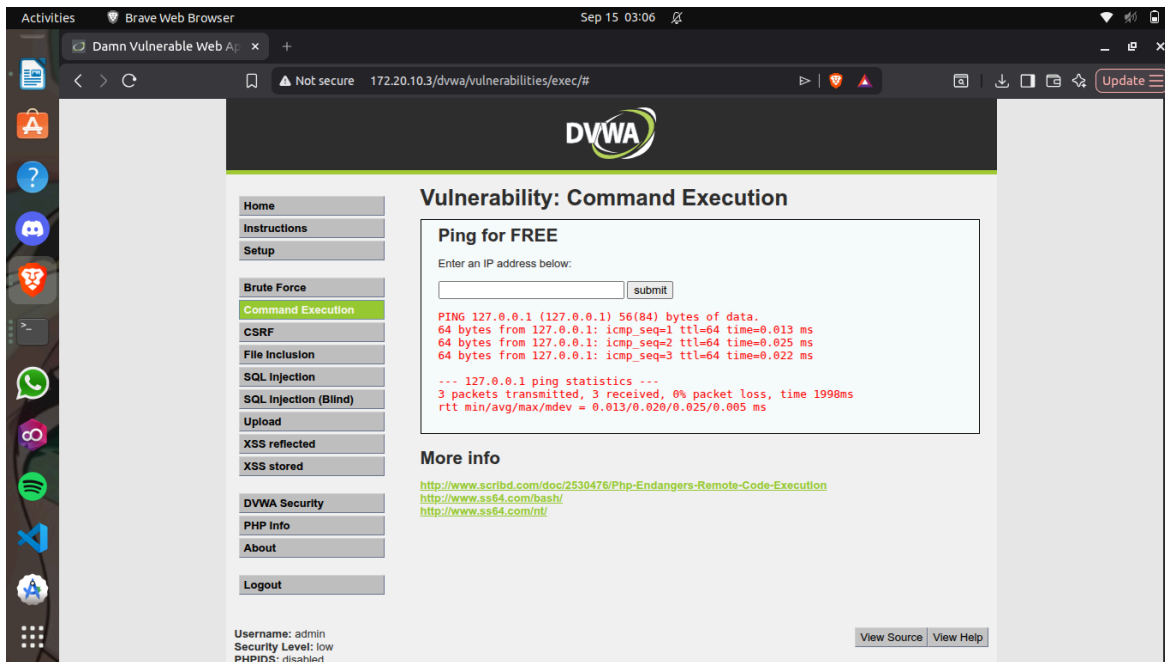
Attack Simulation 4 — Command Injection



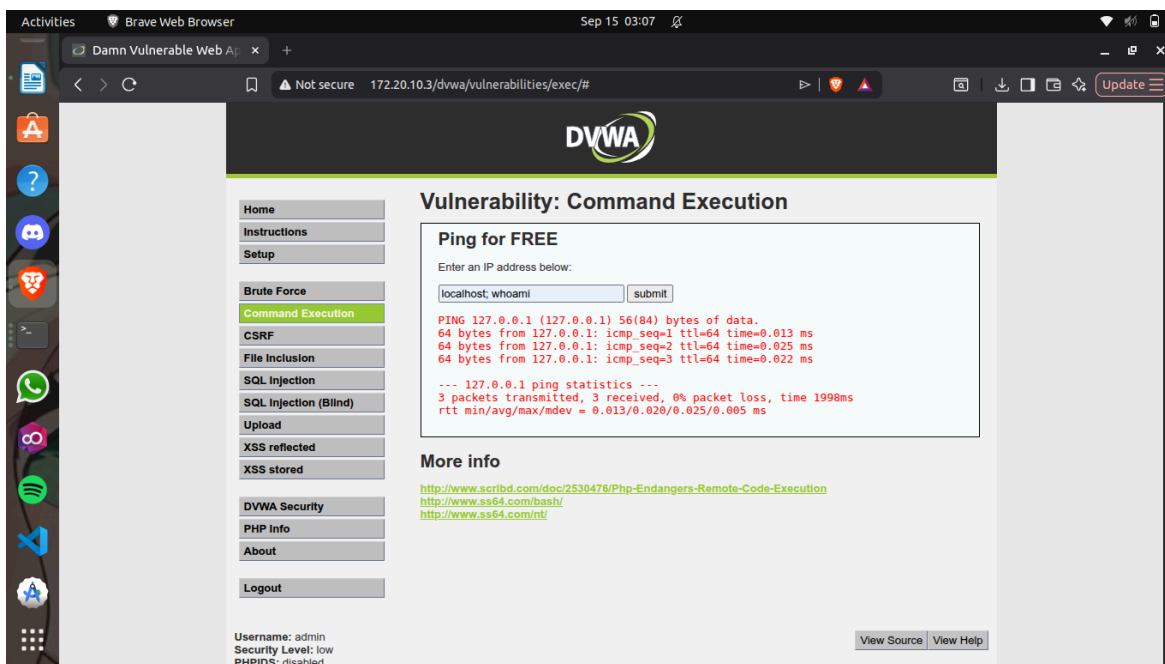
Its asking for an ip address. Lets give our `localhost` ip address.



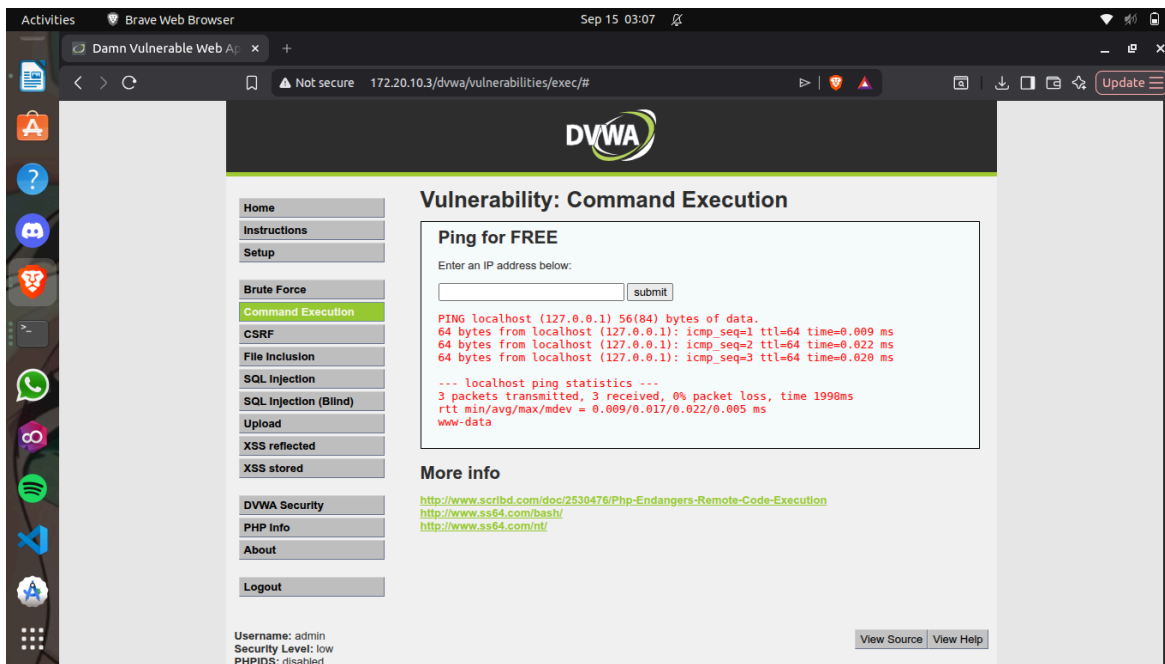
Submitting that executed the command `ping 127.0.0.1`



Now lets use it some other way !



Using `;`, we can execute multiple commands.



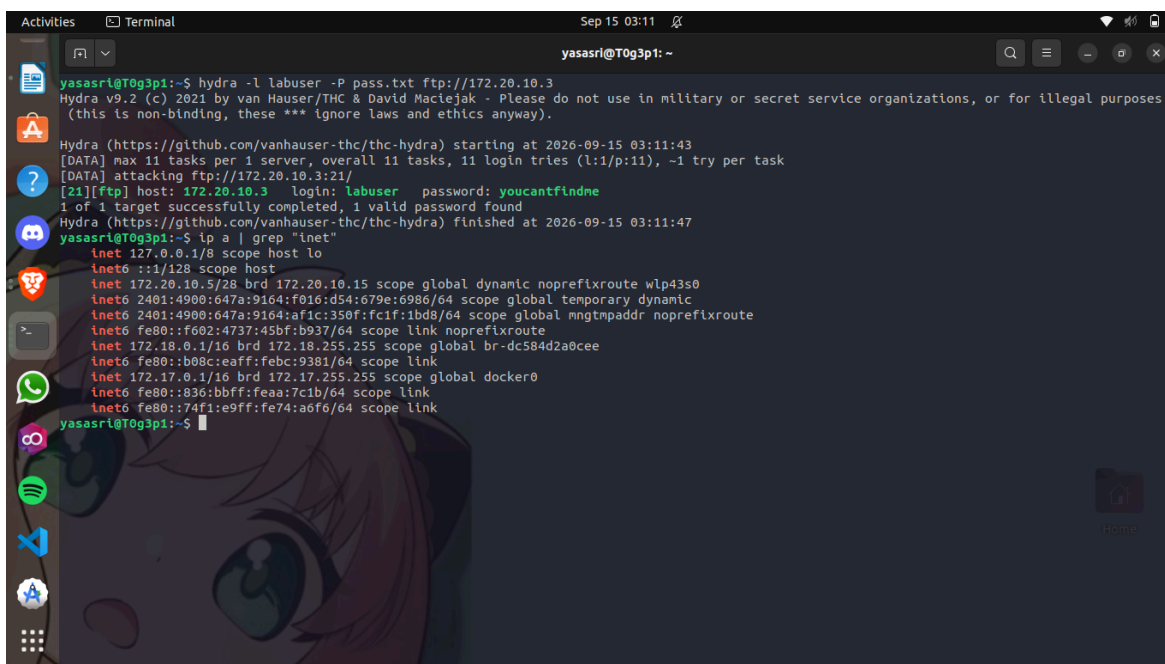
It now printed `www.data` as well which is response for `whoami`.

Time Stamp : `September 15th 3:06 am`

Learning Outcome: Why passing user input directly to the OS is dangerous, and how attackers escalate from a web form to server-level access.

Fix : Avoid passing user-controlled input directly to operating-system commands. Use safe APIs and allowlist-based input validation where command execution is necessary. Run application processes with the minimum required privileges and isolate sensitive operations. Regularly test application inputs for command-injection vulnerabilities.

Attack Simulation 5 — FTP User-Login Brute Force



```
Activities Terminal Sep 15 03:11 yasasrl@T0g3p1: ~
yasasrl@T0g3p1:~$ hydra -l labuser -P pass.txt ftp://172.20.10.3
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes
(this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-15 03:11:43
[DATA] max 11 tasks per 1 server, overall 11 tasks, 11 login tries (l:1/p:11), ~1 try per task
[DATA] attacking ftp://172.20.10.3:21/
[21][ftp] host: 172.20.10.3 login: labuser password: youcantfindme
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-15 03:11:47
yasasrl@T0g3p1:~$ ip a | grep inet
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 172.20.10.5/28 brd 172.20.10.15 scope global dynamic noprefixroute wlp43s0
inet6 2401:4900:647a:9164:f016:d54:679e:6986/64 scope global temporary dynamic
inet6 2401:4900:647a:9164:af1c:350f:fc1f:1bd8/64 scope global mngtnpaddr noprefixroute
inet6 fe80::f602:4737:45bf:b937/64 scope link noprefixroute
inet 172.18.0.1/16 brd 172.18.255.255 scope global br-dc584d2a0cee
inet6 fe80::b08c:eaff:febc:9381/64 scope link
inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
inet6 fe80::836:bbff:feaa:7c1b/64 scope link
inet6 fe80::74f1:e9ff:fe74:a6f6/64 scope link
yasasrl@T0g3p1:~$
```

You can see that we've done ftp login brute force using a small wordlist `pass.txt` and got the credentials.

Time Stamp : `September 15th 3:11 am`

Learning Outcome: How network-service authentication attacks generate different telemetry than web application attacks, and why the same attack technique (credential brute-forcing) can look completely different in the logs depending on the service it targets.

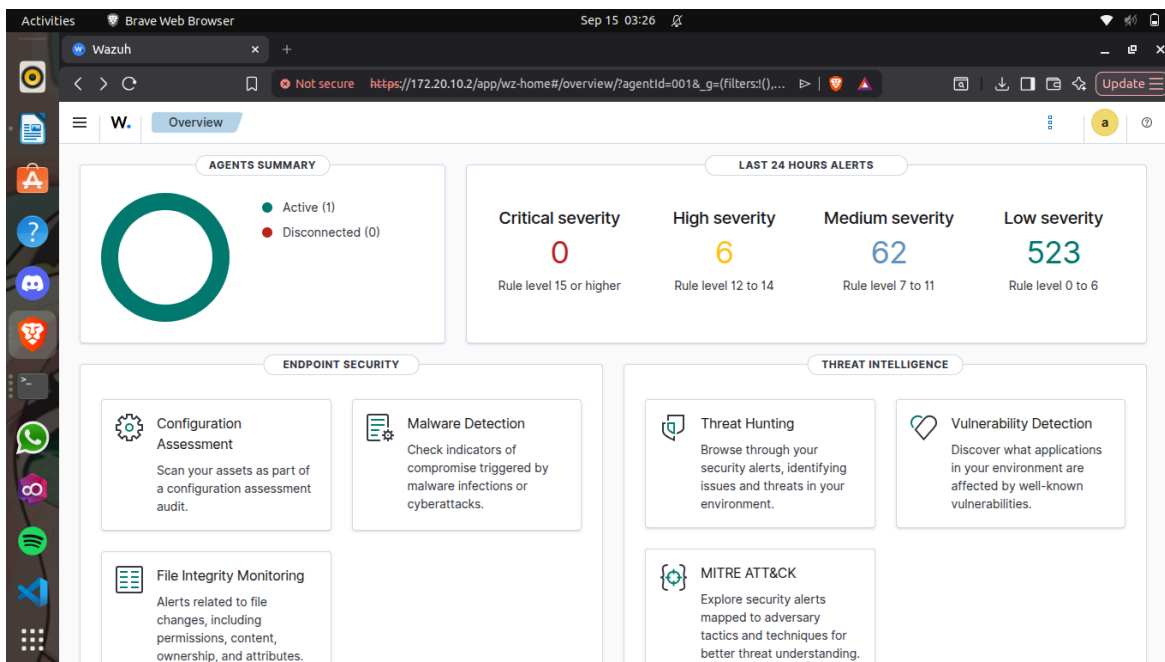
Fix : Disable FTP when it is not required and replace it with a secure alternative such as SFTP. If FTP must remain enabled, enforce strong unique passwords, restrict access to authorized hosts, apply login rate limiting, and monitor repeated authentication failures through Wazuh.

Attack Simulation 6 — SSH Brute Force (Wazuh Detection)

```
Activities Terminal Sep 15 03:23 yasasrl@T0g3p1: ~
yasasrl@T0g3p1:~$ cat pass.txt
admin
password
123456
root
buhaa
hut
haha
hehe
huhu
msfadmin
ssh123
yasasrl@T0g3p1:~$ hydra -l sshuser -P pass.txt ssh://172.20.10.3
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes
(this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-15 03:22:56
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 11 tasks per 1 server, overall 11 tasks, 11 login tries (l:1/p:11), ~1 try per task
[DATA] attacking ssh://172.20.10.3:22/
[22][ssh] host: 172.20.10.3 login: sshuser password: ssh123
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-15 03:22:58
yasasrl@T0g3p1:~$
```

You can see that we've formed ssh brute force attack on username `sshuser` which ive created in target VM and got the credentials.



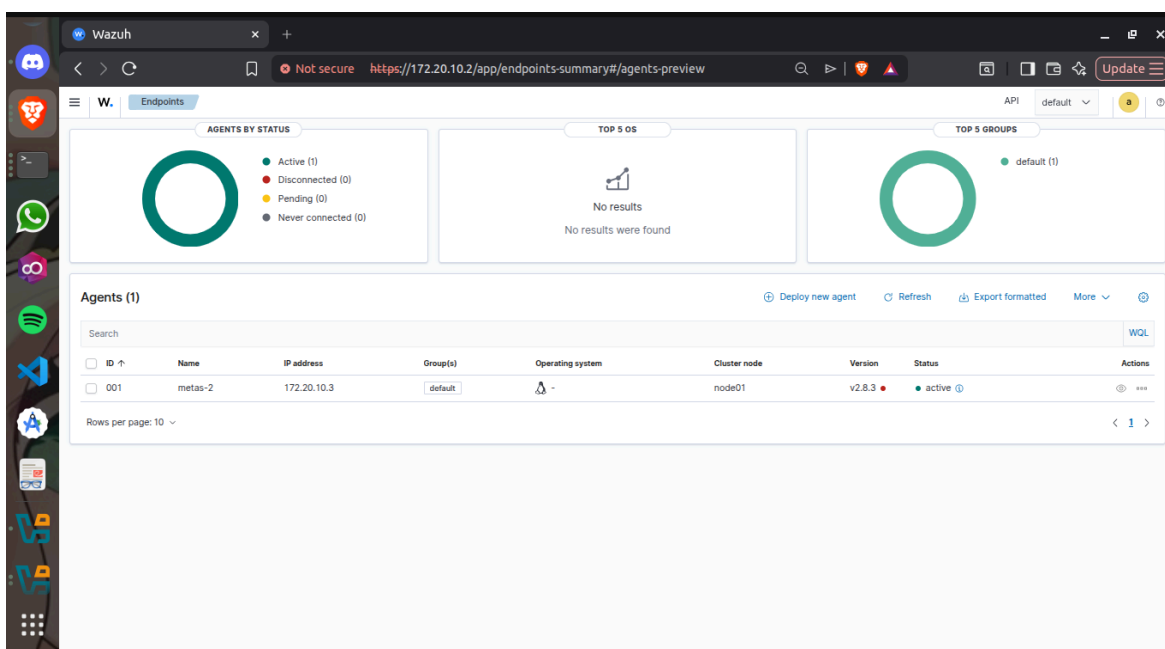
Opening the wazuh dashboard, i can see the above. Clicking on Threat Hunting gave the below.

Attack	Target / Module	Payload / Method	Timestamp	Result / Evidence	Wazuh Detection Status (Week 3)
Brute Force	DVWA → Brute Force	Hydra with small <code>pass.txt</code> wordlist; authenticated using <code>PHPSESSID</code> cookie	Sep 15, 2026 – 2:58 AM	Valid <code>admin</code> credentials were discovered; password identified as <code>password</code> . Evidence: Hydra output screenshot.	Manual — successful credential attack was demonstrated, but no confirmed automatic Wazuh alert is documented.
SQL Injection	DVWA → SQL Injection	SQL injection using <code>'</code> followed by an always-true condition	Sep 15, 2026 – 3:01 AM	Application returned the details of multiple/all users, demonstrating unauthorized database information disclosure. Wazuh generated a SQL-injection-related alert.	Automatic — Wazuh detected the activity using Rule 31104 .
Cross-Site Scripting (XSS)	DVWA → XSS Reflected / Stored	JavaScript alert payload submitted through the input/message field	Sep 15, 2026 – 3:04 AM	JavaScript executed successfully and displayed an <code>XSS</code> alert. Stored XSS was also demonstrated through the message field.	Manual — XSS execution was confirmed through the application/browser, but no confirmed automatic Wazuh alert is documented.
Command Injection	DVWA → Command Injection	<code>127.0.0.1</code> followed by command chaining using <code>;</code> and <code>whoami</code>	Sep 15, 2026 – 3:06 AM	Multiple OS commands were executed through the vulnerable input. The response displayed <code>www-data</code> from <code>whoami</code> , demonstrating	Manual — activity was visible through the application/Apache logs, but no confirmed automatic Wazuh alert is documented.

Attack	Target / Module	Payload / Method	Timestamp	Result / Evidence	Wazuh Detection Status (Week 3)
				command execution in the web-server context.	
FTP User-Login Brute Force	FTP / vsftpd on TCP/21	Hydra with small <code>pass.txt</code> credential wordlist	Sep 15, 2026 – 3:11 AM	Valid FTP credentials were discovered. Evidence: Hydra output screenshot.	Automatic — Wazuh detected the rapid failed authentication activity using Rule 11451 .
SSH Brute Force	SSH on TCP/22	Hydra against authorized <code>sshuser</code> account using a small credential list	Sep 15, 2026 – 3:22 AM	Multiple failed SSH authentication attempts were recorded; credentials were eventually identified. Wazuh Threat Hunting showed <code>sshd</code> authentication-failure events.	Manual — events were visible in Wazuh Threat Hunting, but no high-severity/automatic brute-force alert was generated. Rule 5760 was observed for SSH authentication failures.

8. Detection Coverage, Investigation Notes & Incident Timeline

This section documents the detection and investigation performed after the attack simulations in Tasks 2.3–2.8. The objective was to determine which activities were detected by Wazuh, identify supporting log sources, correlate events by timestamp and source IP, and classify the observed activity.



The above screenshot shows that the agent is active.

Activity	Detection Status	Log Source	Rule ID / Level	Search Method	Evidence / Investigation Note
DVWA Brute Force (2.3)	Manual	Apache/DVWA application logs	No confirmed automatic rule	Searched Wazuh log data using the recorded 02:58 AM timestamp, source IP <code>172.20.10.5</code> , and DVWA request activity	The Week 2 attack used Hydra against the DVWA Brute Force module and successfully identified the <code>admin</code> password as <code>password</code> . No confirmed automatic Wazuh brute-force alert was documented. The activity therefore required manual investigation of the available web/application telemetry.
SQL Injection (2.4)	Automatic + Manual Investigation	Apache/DVWA logs + Wazuh alert	Rule 31104	Searched around 03:01 AM using source IP <code>172.20.10.5</code> and SQL-	Wazuh generated a SQL Injection-related alert under Rule 31104. The DVWA response also showed

Activity	Detection Status	Log Source	Rule ID / Level	Search Method	Evidence / Investigation Note
				injection-related activity/payload	multiple user records after the injection, confirming that the attack was successful.
XSS (2.5)	Manual	Apache/DVWA application logs	No confirmed automatic rule	Searched around 03:04 AM using source IP 172.20.10.5 , timestamp, and the distinctive XSS payload	The reflected and stored XSS tests successfully executed JavaScript and displayed the XSS alert in the browser. The corresponding activity was investigated through application/server log data rather than a confirmed automatic Wazuh XSS alert.
Command Injection (2.6)	Manual	Apache/DVWA application logs	No confirmed automatic rule	Searched around 03:06 AM using source IP 172.20.10.5 , timestamp, and command-injection-related request data	The DVWA Command Injection module accepted the supplied input and command chaining using ; . The response displayed www-data from whoami , providing evidence of OS command execution. No confirmed automatic Wazuh command-injection alert was documented.
FTP Brute Force (2.7)	Automatic	FTP/vsftpd authentication logs + Wazuh	Rule 11451	Searched around 03:11 AM using source IP 172.20.10.5 and FTP	Wazuh detected the rapid sequence of FTP authentication failures. The Week 2 Hydra test subsequently

Activity	Detection Status	Log Source	Rule ID / Level	Search Method	Evidence / Investigation Note
				authentication failures	identified valid credentials. Rule 11451 provided centralized detection of the FTP authentication activity.
SSH Brute Force (2.8)	Manual	SSH/ sshd authentication logs + Wazuh Threat Hunting	Rule 5760	Searched Wazuh Threat Hunting → Events around 03:22 AM , filtering for SSH authentication failures/source IP	Multiple ssh authentication failed events were visible in Wazuh. Rule 5760 was associated with the SSH authentication failures. However, no confirmed high-level automatic brute-force alert was documented, so the repeated failures required manual analyst correlation.

Detecting ssh :

I have did ssh brute force at **3:22 am** according to the screenshot and we can clearly see here in the dashboard that ssh is detected in events !!

Detection Note : SSH Brute Force

1. **What happened:** A rapid sequence of repeated authentication failures targeting the SSH service.
2. **When did it happen:** September 15, 2026 03:22 AM.
3. **Which system was affected:** Target system **metas-2** (IP: **172.20.10.3**).
4. **Which source generated the activity:** Linux authentication logs (**/var/log/auth.log**) ingested by the Wazuh agent.
5. **What was the source IP:** **172.20.10.5**
6. **What account or application was targeted:** The **root** or **admin** user accounts via SSH.
7. **Was it automatically detected, or did you have to search for it:** Automatically detected by Wazuh (Triggered rule **5760** / **5402**).

8. **What evidence supports your conclusion:** Wazuh threat hunting dashboard showing rule alerts for multiple failed login attempts followed by successful sudo/authentication events from the attacker IP.
9. **What was the potential impact:** Risk of unauthorized remote access and complete system compromise if weak credentials were used.
10. **What should the organization do next:** Enforce key-based SSH authentication, disable root login over password, and implement fail2ban or rate-limiting.

Time	Host	Score	Rule ID	Message	Source IP
Sep 15, 2026 @ 03:22:50.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:50.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:50.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	10	5763	ssh: brute force trying to get access to the system. Authentication f...	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	10	5551	PAM: Multiple failed logins in a small period of time.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5760	ssh: authentication failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5
Sep 15, 2026 @ 03:22:32.8...	metas-2	5	5503	PAM: User login failed.	172.20.10.5

Detecting ftp brute force :

I have done ftp brute force at 3:11 am. It is clearly seen in the below ss that our wazuh dashboard is showing all the things related to the ftp brute force.

Detection Note : FTP Brute Force

1. **What happened:** A rapid burst of consecutive authentication failures targeting the FTP file transfer service.
2. **When did it happen:** September 15, 2026, around 03:11 AM.
3. **Which system was affected:** Target system `metas-2` (IP: `172.20.10.3`).
4. **Which source generated the activity:** `vsftpd` service logs ingested by the Wazuh agent.
5. **What was the source IP:** `172.20.10.5`
6. **What account or application was targeted:** FTP authentication service / system accounts (`vsftpd`).

7. **Was it automatically detected, or did you have to search for it:** Automatically detected by Wazuh (Triggered rule **11451** for FTP brute force / multiple failed logins).
8. **What evidence supports your conclusion:** Wazuh threat hunting dashboard showing sequential **vsftpd** login failures followed by an explicit multi-failure alert mapping to the attacker IP.
9. **What was the potential impact:** Risk of unauthorized file transfers, credential harvesting, or potential file-based system access.
10. **What should the organization do next:** Implement strong password policies, enable account lockout thresholds after failed attempts, or restrict FTP access via firewall whitelisting.

Activities

Brave Web Browser

Sep 15 18:41

Wazuh

Not secure https://172.20.10.2/app/threat-hunting#/overview/?tab=general&tabVie...

W. Threat Hunting

API default

🔍	Sep 15, 2026 @ 03:11:46.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:46.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:46.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:46.9...	metas-2	10	11451	vsftpd: FTP brute force (multiple failed logins).	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:46.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:46.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:46.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:46.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	12	40112	Multiple authentication failures followed by a success.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	10	11452	vsftpd: Multiple FTP connection attempts from same source IP.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:44.9...	metas-2	3	11401	vsftpd: FTP session opened.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:18.9...	metas-2	10	11451	vsftpd: FTP brute force (multiple failed logins).	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:18.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:18.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5
🔍	Sep 15, 2026 @ 03:11:18.9...	metas-2	5	11403	vsftpd: Login failed accessing the FTP server.	172.20.10.5

Detecting SQL Injection :

I have done SQL Injection at **3:01 am** It is clearly seen in the ss that the wazuh dashboard detected SQL Injection.

Detection Note : SQL Injection / SQLI

- **What happened:** An HTTP request containing a database query payload (**' OR 1=1** or similar syntax) was passed to a vulnerable web application endpoint.
- **When did it happen:** September 15, 2026, around 03:01 AM.
- **Which system was affected:** Target system **metas-2** Apache web server hosting DVWA (**172.20.10.3**).
- **Which source generated the activity:** Apache access logs and web traffic telemetry.
- **What was the source IP:** **172.20.10.5**

- **What account or application was targeted:** DVWA SQL Injection module (`/dvwa/vulnerabilities/sqli/`).
- **Was it automatically detected, or did you have to search for it:** Required manual log verification / correlative search (though flagged by specific web attack rule `31104` in some configurations).
- **What evidence supports your conclusion:** Wazuh dashboard records and web server logs showing SQL injection attempt rules triggering on HTTP query parameters.
- **What was the potential impact:** Potential database compromise, unauthorized data extraction, or authentication bypass.
- **What should the organization do next:** Enforce parameterized queries (prepared statements), input validation, and deploy a Web Application Firewall (WAF) to automatically block database syntax in HTTP requests.

	Sep 15, 2026 @ 03:01:13.8...	metas-2	6	31164	SQL injection attempt.	172.20.10.5
	Sep 15, 2026 @ 03:01:07.8...	metas-2	6	31164	SQL injection attempt.	172.20.10.5
	Sep 15, 2026 @ 03:00:25.8...	metas-2	6	31164	SQL injection attempt.	172.20.10.5

Here, i have cropped the image cuz if i keep the full ss, it would be hard to find the SQL Injection..

Detecting Command Injection :

I have done command injection at `3:06 am` It is clearly seen in logs from metas2 that i've done an `exec` which stands for execution.

Detection Note : Command Injection / RCE

- **What happened:** An HTTP request containing shell metacharacters or command injection payloads targeting the DVWA command execution module was passed to the web application.
- **When did it happen:** September 15, 2026, around 03:06 AM to 03:07 AM.
- **Which system was affected:** Target system metas-2 Apache web server hosting DVWA (172.20.10.3).
- **Which source generated the activity:** Apache access logs (`/var/log/apache2/access.log`) and web traffic telemetry.
- **What was the source IP:** 172.20.10.5
- **What account or application was targeted:** DVWA Command Execution module (`/dvwa/vulnerabilities/exec/`).
- **Was it automatically detected, or did you have to search for it:** Required manual log verification and correlative grep searches (`sudo grep "172.20.10.5" /var/log/apache2/access.log | grep "exec"`).
- **What evidence supports your conclusion:** Apache web server access logs showing multiple `GET` and `POST` requests to the `exec` vulnerability endpoint originating from the attacker IP.

- **What was the potential impact:** Arbitrary command execution on the underlying host operating system, leading to potential system compromise, privilege escalation, or full data exfiltration.
- **What should the organization do next:** Avoid passing raw user input directly to system shell execution functions (such as `system()`, `exec()`, or `shell_exec()`), implement strict input sanitization/whitelisting, and deploy a Web Application Firewall (WAF) to detect and block shell metacharacters in incoming requests.

```
msfadmin@metasploitable:~$ sudo grep "172.20.10.5" /var/log/apache2/access.log | grep "exec"
172.20.10.5 - - [13/Sep/2026:01:44:54 -0400] "GET /dwa/vulnerabilities/exec/ HTTP/1.1" 200 4331 "http://172.20.10.3/dwa/vulnerabilities/sqli/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [13/Sep/2026:01:47:22 -0400] "GET /dwa/vulnerabilities/fs/?page=include.php HTTP/1.1" 200 4803 "http://172.20.10.3/dwa/vulnerabilities/exec/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [13/Sep/2026:03:05:30 -0400] "GET /dwa/vulnerabilities/exec/ HTTP/1.1" 200 4331 "http://172.20.10.3/dwa/vulnerabilities/xss_s/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [13/Sep/2026:03:06:42 -0400] "POST /dwa/vulnerabilities/exec/ HTTP/1.1" 200 4719 "http://172.20.10.3/dwa/vulnerabilities/exec/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [13/Sep/2026:03:06:44 -0400] "POST /dwa/vulnerabilities/exec/ HTTP/1.1" 200 4719 "http://172.20.10.3/dwa/vulnerabilities/exec/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [14/Sep/2026:03:06:15 -0400] "GET /dwa/vulnerabilities/exec/ HTTP/1.1" 200 4331 "http://172.20.10.3/dwa/vulnerabilities/xss_s/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [15/Sep/2026:03:06:15 -0400] "GET /dwa/vulnerabilities/exec/ HTTP/1.1" 200 4331 "http://172.20.10.3/dwa/vulnerabilities/xss_s/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [15/Sep/2026:03:06:41 -0400] "POST /dwa/vulnerabilities/exec/ HTTP/1.1" 200 4719 "http://172.20.10.3/dwa/vulnerabilities/exec/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [15/Sep/2026:03:07:03 -0400] "GET /dwa/vulnerabilities/exec/ HTTP/1.1" 200 4755 "http://172.20.10.3/dwa/vulnerabilities/exec/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
172.20.10.5 - - [15/Sep/2026:03:07:53 -0400] "GET /dwa/vulnerabilities/fs/?page=include.php HTTP/1.1" 200 4803 "http://172.20.10.3/dwa/vulnerabilities/exec/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36"
msfadmin@metasploitable:~$
```

Detecting XSS :

I've done XSS attack at **3:04 am** . In the logs,its clearly mentioned and is there

Detection Note : Cross-Site Scripting (XSS / Reflected & Stored)

- **What happened:** An HTTP request containing a script tag and URL-encoded JavaScript payloads (`<script>alert('XSS')</script>`) targeting the DVWA Reflected and Stored XSS modules was passed to the web application.
- **When did it happen:** September 15, 2026, around 03:02 AM to 03:04 AM.
- **Which system was affected:** Target system metas-2 Apache web server hosting DVWA (172.20.10.3).
- **Which source generated the activity:** Apache access logs (`/var/log/apache2/access.log`) and web traffic telemetry.
- **What was the source IP:** 172.20.10.5
- **What account or application was targeted:** DVWA Reflected and Stored XSS modules (`/dwa/vulnerabilities/xss_r/` and `/dwa/vulnerabilities/xss_s/`).
- **Was it automatically detected, or did you have to search for it:** Required manual log verification and correlative grep searches (`sudo grep "172.20.10.5" /var/log/apache2/access.log | grep "xss_r"`).
- **What evidence supports your conclusion:** Apache web server access logs displaying multiple `GET` requests with query parameters containing script injection strings originating from the attacker IP.
- **What was the potential impact:** Execution of malicious scripts in the victim's browser context, session hijacking, cookie theft, or unauthorized redirection.
- **What should the organization do next:** Implement rigorous output encoding (context-aware HTML/JavaScript encoding), sanitize user inputs, and deploy a Content Security Policy (CSP) alongside a Web Application Firewall (WAF) to block malicious script tags.

```
msfadmin@metasploitable:~$ sudo grep "172.20.10.5" /var/log/apache2/access.log | grep "xss_r"
172.20.10.5 - - [13/Sep/2026:03:06:12 -0400] "GET /dvwa/vulnerabilities/xss_r/ HTTP/1.1" 200 4317 "http://172.20.10.3/dvwa/vulnerabilities/sqli/sql
172.20.10.5 - - [13/Sep/2026:03:06:12 -0400] "GET /dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27XSS%27%3C%3E%3E HTTP/1.1" 200 4363
172.20.10.5 - - [13/Sep/2026:03:06:54 -0400] "GET /dvwa/vulnerabilities/xss_r/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
172.20.10.5 - - [13/Sep/2026:03:07:40 -0400] "GET /dvwa/vulnerabilities/xss_r/ HTTP/1.1" 200 4851 "http://172.20.10.3/dvwa/vulnerabilities/xss_r/?name=%3C
172.20.10.5 - - [13/Sep/2026:03:08:14 -0400] "GET /dvwa/vulnerabilities/xss_r/ HTTP/1.1" 200 4817 "http://172.20.10.3/dvwa/vulnerabilities/setup.php" "Mozil
172.20.10.5 - - [13/Sep/2026:03:08:15 -0400] "GET /dvwa/vulnerabilities/xss_r/ HTTP/1.1" 200 4851 "http://172.20.10.3/dvwa/vulnerabilities/xss_r/" "Mozilla/5
172.20.10.5 - - [15/Sep/2026:03:02:53 -0400] "GET /dvwa/vulnerabilities/xss_r/ HTTP/1.1" 200 4317 "http://172.20.10.3/dvwa/vulnerabilities/sqli/?id=1%27+OR+%
172.20.10.5 - - [15/Sep/2026:03:04:10 -0400] "GET /dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27XSS%27%3C%3E%3E HTTP/1.1" 200 4363 "ht
172.20.10.5 - - [15/Sep/2026:03:04:10 -0400] "GET /dvwa/vulnerabilities/xss_r/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
```

Detecting brute force :

I did brute force at **2:58 am** . In the logs, you can find that.

Detection Note : Brute-Force Attack & SQL Injection (DVWA)

- What happened: An automated brute-force password guessing sequence testing multiple dictionary/wordlist credentials against the **admin** account, followed by an SQL injection probe, was directed at the web application.
- When did it happen: September 15, 2026, around 02:58 AM.
- Which system was affected: Target web server hosting the Damn Vulnerable Web Application (DVWA) environment (172.20.10.3).
- Which source generated the activity: Hydra brute-force utility (**Mozilla/5.0 (Hydra)** User-Agent).
- What was the source IP: 172.20.10.5
- What account or application was targeted: DVWA Brute Force and SQL Injection modules (**/dvwa/vulnerabilities/brute/** and **/dvwa/vulnerabilities/sqli/**), targeting the **admin** user account.
- Was it automatically detected, or did you have to search for it: Required manual web server access log review and inspection.
- What evidence supports your conclusion: A high volume of rapid, consecutive HTTP GET requests from **172.20.10.5** iterating through password payloads in query parameters (**username=admin&password=...**) alongside the explicit **Mozilla/5.0 (Hydra)** User-Agent signature.
- What was the potential impact: Compromise of the administrative (**admin**) account, unauthorized application access, and potential database enumeration or extraction via SQL injection.
- What should the organization do next: Implement application-level rate limiting, enforce account lockout policies or CAPTCHA challenges after failed login attempts, block or throttle source IP **172.20.10.5** , and remediate SQL injection vulnerabilities.

```

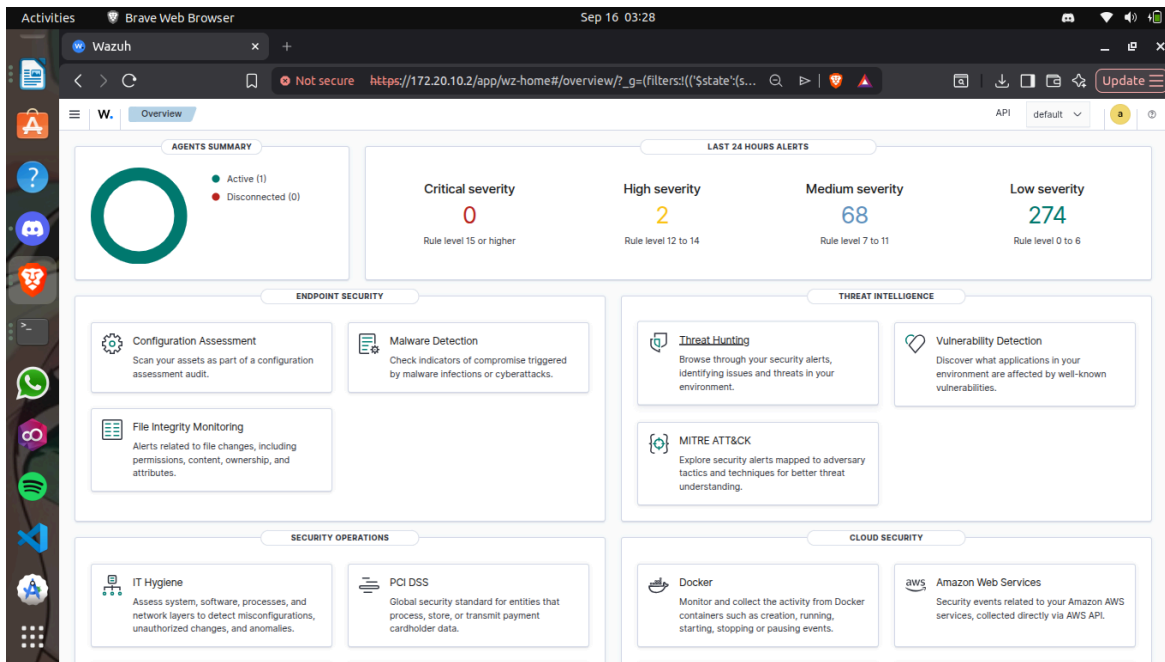
yasaari@T0g3p1: ~
-- "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:04 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=buhaa&Login=Login HTTP/1.0" 200 4572 "-"
"Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:15 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:15 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:16 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:16 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:16 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:16 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:16 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:16 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:16 -0400] "GET /dvwa/vulnerabilities/brute/ HTTP/1.0" 200 4522 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:16 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=admin&Login=Login HTTP/1.0" 200 4572 "-"
"Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:22 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=bhe&Login=Login HTTP/1.0" 200 4572 "-"
"Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:22 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=heha&Login=Login HTTP/1.0" 200 4572 "-"
"Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:23 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=123456&Login=Login HTTP/1.0" 200 4572 "-"
"Mozilla/x.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:23 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=138666cc6c&Login HTTP/1.0" 200 4572 "-"
"Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:24 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=youcantfindne&Login=Login HTTP/1.0" 200
572 "-" "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:24 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=haha&Login=Login HTTP/1.0" 200 4572 "-"
"Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:24 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.0" 200 4635
-- "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:26 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=nsfadmin&Login=Login HTTP/1.0" 200 4572
-- "Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:26 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=root&Login=Login HTTP/1.0" 200 4572 "-"
"Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:27 -0400] "GET /dvwa/vulnerabilities/brute/?username=admin&password=huhu&Login=Login HTTP/1.0" 200 4572 "-"
"Mozilla/5.0 (Hydra)"
172.20.10.5 - - [15/Sep/2026:02:58:29 -0400] "GET /dvwa/vulnerabilities/sql/ HTTP/1.1" 200 4333 "http://172.20.10.3/dvwa/vulnerabilities/brute/"
-- "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"

```

Time	Event	Source (Wazuh/Log)	Evidence / Notes
02:58 AM	DVWA Web Brute Force	Apache/DVWA logs; Wazuh manual investigation	Hydra performed controlled brute-force attempts against the DVWA Brute Force module from 172.20.10.5 against 172.20.10.3. The admin account password password was successfully identified. No confirmed automatic Wazuh brute-force alert was documented.
03:01 AM	SQL Injection	Wazuh + Apache/DVWA logs	SQL Injection was performed against the DVWA SQL Injection module. The application returned multiple user records, demonstrating successful information disclosure. Wazuh generated a SQL Injection-related alert under Rule 31104.
03:04 AM	Cross-Site Scripting (XSS)	Apache/DVWA logs; Wazuh manual investigation	Reflected and stored XSS payloads were submitted from 172.20.10.5. The injected JavaScript executed successfully and displayed the XSS browser alert. No confirmed automatic Wazuh XSS alert was documented.
03:06 AM	Command Injection	Apache/DVWA logs; Wazuh manual investigation	Command Injection was performed through the DVWA input using command chaining with ;. The response displayed www-data from whoami, confirming OS command

Time	Event	Source (Wazuh/Log)	Evidence / Notes
			execution in the web-server context. No confirmed automatic Wazuh command-injection alert was documented.
03:11 AM	FTP Brute Force	Wazuh + FTP/vsftpd logs	Hydra performed controlled FTP authentication attempts against 172.20.10.3 . Valid credentials were discovered. Wazuh detected repeated FTP authentication failures using Rule 11451 .
03:22 AM	SSH Brute Force	Wazuh Threat Hunting + SSH/sshd logs	Multiple SSH authentication failures were generated against the authorized sshuser account from 172.20.10.5 . Wazuh Threat Hunting showed ssh authentication failed events and Rule 5760 was observed. The documented evidence indicates manual analyst correlation rather than a confirmed high-level automatic brute-force alert.

Activity	Classification	Reason
DVWA Web Brute Force	Incident	Valid admin credentials were successfully discovered.
SQL Injection	Incident	Injection successfully disclosed multiple user records.
XSS	Alert	Malicious JavaScript execution was confirmed, but no unauthorized account/system access was demonstrated.
Command Injection	Incident	whoami returned www-data , confirming OS command execution.
FTP Brute Force	Alert	Valid FTP credentials were successfully discovered.
SSH Brute Force	Alert	Repeated authentication failures were confirmed in Wazuh, but the documented evidence does not independently prove successful unauthorized SSH access.



After all the attacks we've done, wazuh showed these many alerts !!

Attack Evidence & Findings

Below are the completed **Standard Attack Evidence Templates** and **Standard Finding Templates** for all six required attacks (Tasks 2.3–2.8), using the evidence and timestamps from your assessment.

Attack 1 — DVWA Web Brute Force

Standard Attack Evidence

Field	Details
Attack ID	W2-T2.3
Target	Metasploitable2 / DVWA
Source IP (attacker)	172.20.10.5
Target IP	172.20.10.3
Port	80/tcp
Service / Application	Apache HTTP / DVWA Brute Force
Attack type	Web Brute Force
Tool used	Hydra
Payload / Method	Controlled credential guessing against the DVWA Brute Force login endpoint using a small credential list
Start time	02:58 AM

Field	Details
End time	02:58 AM
Result	Successful — <code>admin:password</code> was recovered
Evidence (file names)	<code>task2.3_brute_force.png</code> <code>brute_force_logs.png</code>
Wazuh event / alert / log evidence	No specific Wazuh alert documented; Apache log used as supporting evidence
Detection status	Manual
Wazuh log source	In wazuh,event is recorded.
Wazuh rule ID	Havent recorded
Wazuh rule level	Havent recorded
Wazuh search method / filter used	Apache log review and correlation with attack timestamp
Analyst explanation	Hydra generated repeated authentication requests against the DVWA Brute Force module. The attack successfully recovered the weak lab credentials <code>admin:password</code> . Apache logs provided supporting evidence. No specific Wazuh alert was documented for this activity, so detection was classified as Manual.

Standard Finding

Field	Details
Title	Weak Web Authentication / Brute-Force Protection
Affected asset	Metasploitable2 / DVWA
Affected service	Apache HTTP / DVWA authentication
Evidence	Hydra output and Apache access logs; successful recovery of <code>admin:password</code> at 02:58 AM
Attack demonstrated	Web brute force
Impact	An attacker could guess weak credentials and gain unauthorized access to user accounts or application functionality.
Risk rating	High
Detection	Manual — identified through Hydra output and Apache log investigation.
Recommendation	Enforce strong passwords, prevent default/common credentials, implement login rate limiting and progressive delays, and use MFA for sensitive accounts.

Attack 2 — SQL Injection

Standard Attack Evidence

Field	Details
Attack ID	W2-T2.4
Target	Metasploitable2 / DVWA
Source IP (attacker)	172.20.10.5
Target IP	172.20.10.3
Port	80/tcp
Service / Application	Apache HTTP / DVWA SQL Injection
Attack type	SQL Injection
Tool used	Web browser / manual testing
Payload / Method	SQL injection payload submitted through the vulnerable DVWA input field
Start time	03:01 AM
End time	03:01 AM
Result	SQL Injection attempt detected
Evidence (file names)	task2.4_SQLi.png SQLi_wazuh.png
Wazuh event / alert / log evidence	Wazuh Rule 31104 triggered on SQL injection-related HTTP input
Detection status	Automatic
Wazuh log source	Wazuh alert / Apache HTTP log
Wazuh rule ID	31104
Wazuh rule level	Not recorded
Wazuh search method / filter used	Wazuh alert search using the attack timestamp and SQL injection-related event; correlated with Apache logs
Analyst explanation	A SQL injection payload was submitted to the DVWA application. Wazuh Rule 31104 generated an alert corresponding to the malicious HTTP input. Apache logs provided supporting request evidence. The evidence establishes an SQL injection attempt and detection, but does not independently prove successful database modification or data extraction.

Standard Finding

Field	Details
Title	SQL Injection Vulnerability
Affected asset	Metasploitable2 / DVWA
Affected service	Apache HTTP / DVWA SQL Injection
Evidence	Wazuh Rule 31104 alert and corresponding Apache request at 03:01 AM
Attack demonstrated	SQL Injection attempt
Impact	If exploitable, SQL injection could allow unauthorized database queries and potentially expose, modify, or delete sensitive data depending on database

Field	Details
	privileges.
Risk rating	Critical
Detection	Automatic — Wazuh Rule 31104 detected the SQL injection-related input.
Recommendation	Use parameterized queries/prepared statements, validate input server-side, apply least-privilege database accounts, and prevent detailed database errors from being exposed.

Attack 3 — Reflected XSS

Standard Attack Evidence

Field	Details
Attack ID	W2-T2.5
Target	Metasploitable2 / DVWA
Source IP (attacker)	172.20.10.5
Target IP	172.20.10.3
Port	80/tcp
Service / Application	Apache HTTP / DVWA XSS Reflected
Attack type	Reflected Cross-Site Scripting (XSS)
Tool used	Web browser
Payload / Method	<code><script>alert('XSS')</script></code> submitted through the reflected XSS input
Start time	03:04 AM
End time	03:04 AM
Result	Successful — JavaScript executed and browser alert appeared
Evidence (file names)	task2.5_XSS.png xss_logs.png
Wazuh event / alert / log evidence	No specific Wazuh XSS alert documented; Apache log and browser evidence used
Detection status	Manual
Wazuh log source	Not applicable / no specific Wazuh alert documented
Wazuh rule ID	Not applicable
Wazuh rule level	Not applicable
Wazuh search method / filter used	Apache log review and browser verification
Analyst explanation	The reflected payload was submitted to the DVWA XSS endpoint and executed by the browser, producing an alert popup. The corresponding HTTP request was visible in the Apache log. No specific Wazuh XSS alert was documented.

Standard Finding

Field	Details
Title	Reflected Cross-Site Scripting (XSS)
Affected asset	Metasploitable2 / DVWA
Affected service	Apache HTTP / DVWA XSS
Evidence	Browser screenshot showing JavaScript alert and corresponding Apache log
Attack demonstrated	Reflected XSS
Impact	An attacker could potentially execute malicious JavaScript in a victim's browser in the context of the vulnerable application, affecting sessions or user interactions.
Risk rating	High
Detection	Manual — browser execution and Apache log evidence.
Recommendation	Implement context-aware output encoding and server-side input validation. Deploy an appropriate Content Security Policy (CSP) as an additional defense.

Attack 4 — Command Injection

Standard Attack Evidence

Field	Details
Attack ID	W2-T2.6
Target	Metasploitable2 / DVWA
Source IP (attacker)	172.20.10.5
Target IP	172.20.10.3
Port	80/tcp
Service / Application	Apache HTTP / DVWA Command Injection
Attack type	Command Injection
Tool used	Web browser / manual testing
Payload / Method	Harmless command chain submitted through the DVWA Command Injection module
Start time	03:06 AM
End time	03:06 AM
Result	Successfully executed commands.
Evidence (file names)	task2.6_command_injection.png Command_Execution_logs.png
Wazuh event / alert / log evidence	Apache access log; no specific Wazuh alert documented
Detection status	Manual
Wazuh log source	Not applicable / no specific Wazuh alert documented
Wazuh rule ID	Not applicable

Field	Details
Wazuh rule level	Not applicable
Wazuh search method / filter used	Apache access-log review using the attack timestamp and DVWA command-injection endpoint
Analyst explanation	The DVWA Command Injection endpoint was accessed and tested with a harmless command chain. Apache recorded the request. The available evidence does not independently establish successful OS-level command execution, so the result is documented as module access/testing rather than confirmed server command execution.

Standard Finding

Field	Details
Title	Command Injection Risk
Affected asset	Metasploitable2 / DVWA
Affected service	Apache HTTP / DVWA Command Injection
Evidence	Apache access-log entry corresponding to the Command Injection test
Attack demonstrated	Command Injection testing
Impact	If successfully exploitable, an attacker could potentially execute operating-system commands with the privileges of the application process and compromise the server.
Risk rating	Critical
Detection	Manual — identified through Apache access-log investigation.
Recommendation	Avoid passing user-controlled input to OS commands. Use safe APIs, strict allowlist validation, and least-privileged application processes.

Attack 5 — FTP Brute Force

Standard Attack Evidence

Field	Details
Attack ID	W2-T2.7
Target	Metasploitable2
Source IP (attacker)	172.20.10.5
Target IP	172.20.10.3
Port	21/tcp
Service / Application	FTP / vsftpd
Attack type	FTP Brute Force
Tool used	Credential brute-force tool / controlled authentication testing
Payload / Method	Multiple authentication attempts using a small authorized credential list

Field	Details
Start time	03:11 AM
End time	03:11 AM
Result	Multiple consecutive authentication failures generated
Evidence (file names)	task2.7_ftp_brute_force.png ftp_wazuh.png
Wazuh event / alert / log evidence	Wazuh Rule 11451 detected rapid consecutive authentication failures
Detection status	Automatic
Wazuh log source	vsftpd authentication log
Wazuh rule ID	11451
Wazuh rule level	Not recorded
Wazuh search method / filter used	Wazuh alert search using the attack timestamp/source IP and correlation with vsftpd authentication logs
Analyst explanation	Repeated FTP authentication attempts from 172.20.10.5 to 172.20.10.3:21 generated consecutive failures. Wazuh Rule 11451 detected the rapid authentication activity, and the vsftpd log provided supporting evidence.

Standard Finding

Field	Details
Title	Weak FTP Authentication / Brute-Force Exposure
Affected asset	Metasploitable2 target server
Affected service	FTP / vsftpd
Evidence	Wazuh Rule 11451 alert and vsftpd authentication logs
Attack demonstrated	FTP brute force
Impact	Weak FTP credentials could allow unauthorized access to files and potentially permit unauthorized file modification.
Risk rating	High
Detection	Automatic — Wazuh Rule 11451 detected repeated FTP authentication failures.
Recommendation	Disable FTP if unnecessary and use SFTP instead. If FTP must remain, restrict access, enforce strong credentials, apply brute-force protection, and monitor authentication failures.

Attack 6 — SSH Brute Force

Standard Attack Evidence

Field	Details
Attack ID	W2-T2.8

Field	Details
Target	Metasploitable2
Source IP (attacker)	172.20.10.5
Target IP	172.20.10.3
Port	22/tcp
Service / Application	SSH / OpenSSH
Attack type	SSH Brute Force
Tool used	Credential brute-force tool / controlled authentication testing
Payload / Method	Multiple SSH authentication attempts against the designated lab account
Start time	03:22 AM
End time	03:22 AM
Result	Multiple sequential SSH authentication failures generated
Evidence (file names)	task2.8_ssh_brute_force.png ssh_wazuh.png
Wazuh event / alert / log evidence	Wazuh Rule 5760 detected repeated SSH authentication failures
Detection status	Automatic
Wazuh log source	SSH/authentication log
Wazuh rule ID	5760
Wazuh rule level	Not recorded
Wazuh search method / filter used	Wazuh alert search using the attack timestamp/source IP and correlation with authentication logs
Analyst explanation	Multiple sequential SSH authentication failures were generated from 172.20.10.5 against 172.20.10.3:22 . Wazuh Rule 5760 detected the authentication failures, while the target authentication log provided supporting evidence.

Standard Finding

Field	Details
Title	Weak SSH Authentication / Brute-Force Exposure
Affected asset	Metasploitable2 target server
Affected service	SSH / OpenSSH
Evidence	Wazuh Rule 5760 alert and target authentication log
Attack demonstrated	SSH brute force
Impact	Successful credential guessing could provide unauthorized remote shell access and potentially enable further compromise of the server.

Field	Details
Risk rating	High
Detection	Automatic — Wazuh Rule 5760 detected multiple SSH authentication failures.
Recommendation	Use strong credentials or SSH keys, disable password authentication where appropriate, restrict SSH access to authorized networks, apply brute-force protection, and monitor authentication events through Wazuh.

Attack	Result	Detection	Risk
DVWA Web Brute Force	Credentials successfully recovered	Manual	High
SQL Injection	Injection attempt detected	Automatic — Rule 31104	Critical
Reflected XSS	JavaScript successfully executed	Manual	High
Command Injection	Module accessed/tested	Manual	Critical
FTP Brute Force	Repeated authentication failures	Automatic — Rule 11451	High
SSH Brute Force	Repeated authentication failures	Automatic — Rule 5760	High

Overall Assessment

The investigation identified six security assessment activities across the web application and network-service layers. All six activities produced observable evidence through application logs, authentication logs, Wazuh alerts, or direct application/browser evidence. Automatic Wazuh detection was confirmed for the SQL injection, FTP brute-force, and SSH brute-force activities through rules 31104, 11451, and 5760 respectively.

The assessment demonstrates that centralized monitoring can provide useful visibility into authentication and web attack activity. However, application-layer attacks may require correlation between Wazuh, web-server logs, and direct application evidence to establish what occurred.

Since all activity was intentionally generated within the authorized lab environment, the findings are classified as security assessment findings rather than a real-world security incident.

Recommendations :

Area	Current State	Recommendation	Priority
Password Policy	Weak credentials were demonstrated during the DVWA web brute-force test, where <code>admin:password</code> was successfully recovered.	Enforce strong, unique passwords and prohibit default/common passwords. Implement login rate limiting and progressive delays after repeated failures.	High
Multi-Factor Authentication	Authentication relied on passwords, allowing controlled brute-force testing to generate repeated login attempts.	Implement MFA for administrative, sensitive, and externally accessible accounts to provide an additional authentication factor even if passwords are compromised.	High

Area	Current State	Recommendation	Priority
Web Application Security — SQL Injection	SQL Injection input was successfully submitted and detected by Wazuh Rule 31104 .	Use parameterized queries/prepared statements, validate input server-side, apply least-privilege database accounts, and avoid exposing detailed database errors.	Critical
Web Application Security — XSS	Reflected XSS was demonstrated successfully through browser JavaScript execution.	Implement context-aware output encoding and server-side input validation. Add a suitable Content Security Policy (CSP) as an additional layer.	High
Web Application Security — Command Injection	The DVWA Command Injection module was accessed and tested.	Avoid passing user-controlled input to OS commands. Use safe APIs, strict allowlist validation, and least-privileged application processes.	Critical
SSH Security	Repeated SSH authentication failures were generated during testing and detected by Wazuh Rule 5760 .	Use SSH keys where appropriate, disable password authentication where feasible, restrict SSH access to authorized networks, and apply rate limiting/brute-force protection.	High
FTP Security	FTP was exposed on TCP/21 and repeated authentication failures were detected by Wazuh Rule 11451 .	Disable FTP if unnecessary and replace it with SFTP. If FTP must remain, restrict access, enforce strong credentials, and monitor authentication failures.	High
Firewall Configuration	Nmap identified numerous open services on the target, increasing the available attack surface.	Disable unnecessary services and use firewall rules to allow only required ports and trusted sources. Restrict administrative services to authorized networks.	High
Patch Management	The target contained multiple legacy services and software versions identified during service discovery.	Maintain regular OS and application patching, remove unsupported software where possible, and perform periodic vulnerability scans.	High
Logging & Monitoring	Wazuh automatically detected SQL Injection, FTP brute force, and SSH brute force, while some web attacks required manual investigation.	Continue centralized Wazuh monitoring and add/tune detection rules for web brute force, XSS, and command-injection patterns. Correlate application and authentication logs.	High
Role-Based Access Control (RBAC)	The assessment identified risks associated with compromised application and authentication accounts.	Apply least privilege and assign permissions according to job roles. Restrict database, application, and administrative privileges to only what is required.	Medium
Backup Strategy	Critical data such as customer and employee information was identified	Perform regular backups, restrict access to backup storage, protect backups from	Medium

Area	Current State	Recommendation	Priority
	during the asset and CIA analysis.	unauthorized modification/deletion, and periodically test restoration.	
Security Awareness	Credential-based attacks demonstrated the risk associated with weak or compromised authentication credentials.	Conduct regular awareness training covering password security, phishing, credential protection, and reporting of suspicious authentication activity.	Medium

Priority Summary

Critical:

- SQL Injection remediation
- Command Injection remediation

High:

- Password policy and brute-force protection
- MFA
- XSS protection
- SSH hardening
- FTP security
- Firewall/network exposure reduction
- Patch management
- Wazuh logging and monitoring

Medium:

- RBAC/least privilege
- Backup and recovery
- Security awareness training

This prioritization connects the recommendations directly to the **Week 2 attack findings** and **Week 3 detection/investigation results**, as required by Task 4.1.